

Interacción juego-teórica para la predicción de vínculos y detección de comunidades

TRABAJO FIN DE GRADO

Curso 2024/2025



UNIVERSIDAD COMPLUTENSE DE MADRID
FACULTAD DE CIENCIAS MATEMÁTICAS

Grado en Matemáticas

Natalia Monsalve Baeza

Tutor: Juan Antonio Tejada Cazorla

Madrid, 20 de febrero de 2025.

Resumen

Una red social se define como una estructura de interacciones en la que los nodos representan agentes y las aristas modelan las relaciones entre ellos. El auge de las redes sociales en línea en las últimas décadas ha generado una gran cantidad de información disponible, lo que ha permitido una investigación más profunda sobre su estructura y dinámica.

Dentro de la exploración de datos, existen enfoques que analizan los vínculos y la estructura de la red, facilitando la identificación de relaciones clave y comunidades relevantes. Este análisis puede abordarse a través del concepto de *interacción* definido en teoría de juegos cooperativos. Los índices de interacción, derivados de conceptos de solución en teoría de juegos, se emplearán como medida de similitud entre nodos, proporcionando una herramienta eficaz para la predicción de vínculos y detección de comunidades.

En este trabajo se introducirán los conceptos fundamentales de teoría de juegos cooperativos que permiten la formulación de distintos índices de interacción, a partir de los cuales se diseñarán algoritmos para su cálculo, los cuales serán aplicados a los problemas de predicción de vínculos y detección de comunidades en conjuntos de datos reales. Finalmente, los resultados obtenidos se compararán con otras medidas de similitud locales para evaluar su eficacia.

Palabras clave

Índices de interacción, valor de Shapley, teoría de juegos cooperativa, predicción de vínculos, detección de comunidades.

Abstract

Social networks are defined as structures of interactions in which nodes represent actors and edges model the relationships between them. The rise of online social networks in recent decades has provided us with large amounts of data, enabling more in-depth research into their structure and dynamics.

From the perspective of data mining, two primary types of data can be distinguished. One of them is linkage-based and structural analysis, which allows us to identify important communities and connections between nodes. This analysis can be approached through the concept of interaction, as defined in cooperative game theory. Interaction indices, derived from solution concepts in game theory, will be employed as measures of similarity between nodes, providing an effective tool for link prediction and community detection.

This study introduces the fundamental concepts of cooperative game theory, which serve as the basis for the formulation of various interaction indices. Based on these indices, we will design algorithms for their computation and apply them to the problems of link prediction and community detection in real-world datasets. Finally, the obtained results will be compared with other local similarity measures to assess the efficiency of our approach.

Keywords

Interaction indices, Shapley value, cooperative game theory, link prediction, community detection.

Agradecimientos

Me gustaría expresar mi agradecimiento a todas aquellas personas que han formado parte de mi formación en el mundo de las matemáticas. En particular, al profesorado de la Facultad de Ciencias Matemáticas de la Complutense, institución que me ha enseñado mucho, no solo a través de sus docentes, sino también gracias a las personas que la habitan. Gracias a ellos, cuatro años después, puedo decir con certeza que elegir las matemáticas fue la decisión correcta.

También quiero agradecer a mi familia, sin la que no hubiese podido tomar el camino que me ha traído hasta aquí, y a mis amigos, en especial a las amistades que esta carrera me ha brindado: Carmen, María, Alejandro e Irene.

No puedo dejar de mencionar a quienes despertaron en mi el interés por esta ciencia, en especial a mi profesor Antonio.

Por último, le quiero dar las gracias a mi tutor Juan Tejada, por su paciencia, dedicación y guía a lo largo de este proceso. Su apoyo ha sido fundamental para la realización de este trabajo.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Antecedentes	1
1.3. Objetivos	2
1.4. Estructura del documento	2
2. Preliminares	3
2.1. Introducción a la teoría de juegos cooperativos	3
2.2. Valores e Índices de interacción cooperativos	5
2.2.1. Valor de Shapley	7
2.2.2. Valor de Banzhaf	9
2.2.3. Valores probabilísticos, semivalores	9
2.2.4. Índices de interacción	10
2.3. Teoría de grafos	12
3. Índices de interacción en redes	13
3.1. Planteamiento del problema	13
3.2. Índice de interacción de Szczepáński	16
3.3. Índice de interacción de Tarkowski	20
4. Predicción de enlaces	25
4.1. Evaluación del rendimiento en predicción de enlaces	25
4.1.1. Medidas de similitud para la comparación	26
4.1.2. Métricas de eficiencia	26
4.1.3. Diseño experimental	27
4.2. Experimento por etapas	28
5. Detección de comunidades	30
5.1. Clustering jerárquico e índices de interacción	30
5.2. Evaluación del rendimiento en detección de comunidades	32
5.2.1. Algoritmos de comparación	32
5.2.2. Métricas de eficiencia	33
5.2.3. Diseño experimental	34
6. Resultados experimentales y conclusiones	36
6.1. Predicción de vínculos	36
6.2. Detección de comunidades	38

A. Anexo I: Código de Python	43
A.1. Herramientas	43
A.2. Problema inicial de predicción de vínculos	48
A.3. Problema de reclasificación	63
A.4. Problema de detección de comunidades	69

Índice de figuras

6.1. Particiones grafo Zachary en dos comunidades	39
6.2. Particiones grafo Zachary comparación Louvain	40
6.3. Proceso de clustering aglomerativo para SVMII y SVII con $k=3$	40

Capítulo 1

Introducción

1.1. Motivación

A lo largo de la historia, las matemáticas han evolucionado como respuesta a los desafíos que emergen del mundo natural y las interacciones humanas.

Dentro del marco de las matemáticas, el **análisis de redes sociales** nació de la necesidad de entender la estructura y el comportamiento de sistemas interconectados. Mientras que, por otro lado, la teoría de juegos surgió con el fin de modelar el conflicto y la cooperación entre agentes racionales, estableciendo unas normas que permiten responder preguntas fundamentales sobre la toma de decisiones estratégicas.

Con el auge de las redes sociales en línea y la creciente disponibilidad de datos sobre todo tipo de interacciones, han cobrado especial interés dos problemas dentro del análisis de redes dinámicas: la **predicción de vínculos**, que busca anticipar futuras conexiones en la red, y la **detección de comunidades**, que pretende identificar grupos de nodos con patrones de interacción más intensos.

Este trabajo surge del interés por abordar estos problemas desde un enfoque *juego-teórico*, integrando dos disciplinas vinculadas: la teoría de redes y la teoría de juegos, ambas centradas en el estudio de las dinámicas de interacción. En particular, se emplearán conceptos de teoría de juegos cooperativa, como son los índices de interacción, en el contexto de *juegos de influencia*, donde cada agente racional busca maximizar su influencia en la red. Estos índices actuarán como una medida de similitud entre grupos de vértices, integrándose dentro de las estrategias basadas en semejanza para la predicción de vínculos y detección de comunidades.

1.2. Antecedentes

En esta sección mencionamos los trabajos de referencia que han sido clave para el desarrollo de este estudio. Aunque la mayoría pertenecen a una única disciplina, como es la teoría de juegos cooperativos, destacan dos artículos pioneros en la fusión de los conceptos de redes y teoría de juegos cooperativos. El primero, propuesto por Szczepanski et al. (2015), introdujo la aplicación de los índices de interacción en la predicción de vínculos y detección de comunidades. Posteriormente, en Tarkowski et al. (2019) se refinó esta propuesta con el objetivo de mejorar la precisión en la predicción de vínculos.

La década de oro en el campo de la *Teoría de Juegos* se sitúa entre los años 50 y 60. Durante este periodo, no solo tuvo lugar el origen formal de esta teoría con la obra de von Neumann & Morgenstern (1944), sino que también surgieron muchos de los conceptos fundamentales de esta

disciplina. Entre ellos, destaca la noción de *solución*, introducida por Lloyd Shapley en Shapley (1953), de la que deriva todo nuestro estudio. Este matemático y economista fue el creador del concepto de solución más empleado en teoría de juegos cooperativos, el valor de Shapley (1953). En este trabajo, también usaremos el valor de Banzhaf III (1965) y el concepto de índice de interacción, formulado por primera vez por Owen (1972). Este último constituye el eje central de este trabajo y ha sido ampliamente analizado por distintos autores, entre los que resalta la extensión axiomática de los valores de Shapley y Banzhaf publicada por Grabisch & Roubens (1999).

Dentro del área del análisis de redes sociales, el concepto de similitud entre vértices no es un problema nuevo. Jaccard propuso un índice de similitud entre dos conjuntos en el año 1901, el cual se aplicó a teoría de redes considerando intersecciones de vecindades. Las primeras formulaciones de los problemas de predicción de vínculos y detección de comunidades son más recientes. El primero fue estudiado formalmente por Liben-Nowell & Kleinberg (2003), mientras que el segundo fue introducido por Girvan & Newman (2002) propuso además, dos años más tarde, la modularidad como medida para la evaluación de la calidad de las particiones de un grafo.

1.3. Objetivos

La finalidad de este trabajo es, en primer lugar, unificar e integrar los enfoques previos de Szczepanski et al. (2015) y Tarkowski et al. (2019) en el marco de los índices de interacción definidos sobre redes sociales, proporcionando una notación única, clara y comprensible.

Asimismo, este estudio tiene como objetivo ampliar el análisis de los trabajos previamente mencionados mediante la aplicación de distintos índices de interacción a los problemas fundamentales: la predicción de vínculos y detección de comunidades. A partir de los resultados obtenidos, se extraerán conclusiones sobre la eficacia de estas medidas de similitud locales en redes, destacando sus ventajas y limitaciones en diferentes escenarios.

1.4. Estructura del documento

Este trabajo está organizado en seis capítulos. En el segundo se incluirá una amplia base teórica sobre teoría de juegos cooperativos, proporcionando los conceptos esenciales, junto con las definiciones básicas de grafos, que constituirán la base de nuestras redes de trabajo.

En el Capítulo 3, se formularán los índices de interacción empleados en los problemas de redes dinámicas planteados. Además, se presentarán ciertas demostraciones constructivas que facilitarán la comprensión de las expresiones de estas medidas de similitud.

Los problemas de predicción de vínculos y detección de comunidades se abordarán en los Capítulos 4 y 5, respectivamente. La estructura seguida incluirá secciones en las que introduciremos las métricas de evaluación para analizar la eficiencia de los índices de interacción, así como otras metodologías con las que compararlas. También se detallará el procedimiento experimental seguido en ambos casos. Como novedad, se propondrá un experimento adicional en el campo de la predicción de enlaces.

Finalmente, el Capítulo 6 realizará un recopilatorio de los resultados obtenidos más relevantes y expondrá una conclusión acerca del rendimiento de los índices de interacción como medidas de semejanza en los dos problemas principales de redes dinámicas.

Capítulo 2

Preliminares

2.1. Introducción a la teoría de juegos cooperativos

La teoría de juegos es una disciplina matemática que se centra en el análisis y la modelización de situaciones de conflicto y cooperación entre decisores racionales e inteligentes. En estas situaciones, llamadas **juegos**, cada jugador busca la estrategia óptima con la que conseguir sus objetivos, y en base a dicha estrategia, tomar decisiones. El escenario general de un juego permite a todo jugador conocer el conjunto de personas que participan en él y sus preferencias para la búsqueda de una estrategia.

En los juegos se puede permitir o no la comunicación entre jugadores. El hecho de poder comunicarse entre participantes da lugar a que se establezcan entre ellos negociaciones o acuerdos vinculantes por medio de una serie de mecanismos establecidos. Los juegos que permiten cooperación entran dentro del marco de la **teoría de juegos cooperativa**. En ellos se considera información básica el beneficio que cada coalición puede obtener coordinando las estrategias de sus miembros, con independencia del resto de participantes del juego.

Dentro del marco de la teoría de juegos cooperativos, un juego no se tratará de una competición entre individuos, sino más bien entre posibles coaliciones emergentes. Al formar coaliciones, los agentes actuarán de manera conjunta como si fueran un único jugador racional, obteniendo la máxima utilidad conjunta. Así, el problema central de estudio radica en la distribución de esta utilidad.

A continuación introduciremos los juegos cooperativos a partir de la formulación de su *función característica*, la cual asigna a cada coalición la utilidad o beneficio que puede llegar a obtener si sus miembros cooperan. Estos juegos se denominan **juegos de reparto del beneficio**, y serán los únicos con los que trabajaremos.

Definición 2.1.1. Sean:

- Un conjunto finito de jugadores denotado por $N = \{1, \dots, n\}$.
- Una función característica $v : 2^N \rightarrow \mathbb{R}$ que asocia a cada subconjunto $S \subseteq N$ (o coalición) un número real $v(S)$ que indica su valor, con $v(\emptyset) = 0$. Esta función también se conoce con el nombre de **función de utilidad**.

Decimos que la tupla $G = (N, v)$ es un juego cooperativo.

Observación 2.1.1. La función $v : 2^N \rightarrow \mathbb{R}$ no determina cómo debe repartirse entre sus integrantes el beneficio ni impone restricciones sobre su distribución. En los casos en los que la utilidad

puede distribuirse libremente entre los jugadores, hablamos de juegos de **utilidad transferible** (*TU games*).

Nota. Con el fin de evitar una notación excesivamente cargada, adoptaremos la siguiente convención:

- Omitiremos el uso de llaves en el caso de los conjuntos unitarios, escribiendo $v(i)$ en lugar de $v(\{i\})$ y $U \setminus i$ en lugar de $U \setminus \{i\}$.
- Representaremos los pares como ij en lugar de $\{i, j\}$.
- Usaremos letras en minúsculas para denotar la cardinalidad de conjuntos, por ejemplo, $s = |S|$.
- En el caso en que no exista ambigüedad sobre N , identificaremos el juego (N, v) con su función característica v .

A continuación, introduciremos algunas propiedades de juegos cooperativos que caracterizan la relación entre las coaliciones y las situaciones que modelizan.

Definición 2.1.2. Un juego cooperativo $G = (N, v)$ se dice que:

- Es **superaditivo** si, para cualesquiera coaliciones $S, T \subseteq N$, con $S \cap T = \emptyset$, se cumple:

$$v(S) + v(T) \leq v(S \cup T)$$

En este caso la formación de la coalición total resulta ventajosa en términos de utilidad obtenida.

- Es **subaditivo** si, para cualesquiera coaliciones $S, T \subseteq N$, con $S \cap T = \emptyset$, se cumple:

$$v(S) + v(T) \geq v(S \cup T)$$

- Es **subaditivo débil** si se satisface que:

$$\forall i \in N, S \subseteq N \setminus i : v(S \cup i) \leq v(S) + v(i)$$

En estos dos últimos tipos de juego no existe beneficio en la fusión de coaliciones.

- Es **monótono** si, para cualesquiera dos coaliciones $S, T \subseteq N$, con $S \subseteq T$, se verifica que:

$$v(S) \leq v(T)$$

- Es **aditivo** si, para cualesquiera dos coaliciones $S, T \subseteq N$, tales que $S \cap T = \emptyset$ se tiene que:

$$v(S \cup T) = v(S) + v(T)$$

- Es **simple** si, además de ser monótono, cumple que $v(S) \in \{0, 1\} \forall S \subseteq N$ y $v(N) = 1$.

Nota. Los juegos cooperativos simples son usados para la modelización de sistemas de decisión. En este contexto, los juegos simples describen la capacidad de decisión de las coaliciones en un sistema de votación. De manera que una coalición $S \subseteq N$ se dice que es **ganadora** si $v(S) = 1$ y, en el caso en que tenga utilidad cero, decimos que es **perdedora**.

Definición 2.1.3. Sea $v : 2^N \rightarrow \mathbb{R}$ un juego simple, $S \subseteq N, i \in S$, decimos que el jugador i es un **pivote** en la coalición S si su presencia es decisiva para que la coalición sea ganadora. Es decir, si $v(S) = 1$, pero $v(S \setminus i) = 0$.

Pensemos que en un mecanismo decisorio, el *poder* viene a ser como una unidad de beneficio que hay que repartir entre los agentes de forma que refleje la importancia estratégica de cada uno. Las medidas que realizan este reparto de poder son conocidas como **índices de poder**, los cuales tendrá en cuenta en cuántas coaliciones un jugador es *pivote*.

El conjunto de juegos cooperativos con utilidad transferible cuyo conjunto de jugadores es N se representa por $\mathcal{G}_N$. Este tiene estructura de espacio vectorial sobre el cuerpo de los números reales, con dimensión $2^n - 1$. Un aspecto clave en estos juegos es el reparto del beneficio, ya que influye en la decisión de los jugadores de unirse o abandonar una coalición.

Definición 2.1.4. Dado un conjunto de jugadores $N = \{1, \dots, n\}$, una **propuesta de pagos** es un vector $x = (x_1, \dots, x_n) \in \mathbb{R}^n$, donde x_i representa el pago que recibe el jugador i por el vector x .

Un vector de pagos no es más que una propuesta de distribución de los beneficios derivados de la cooperación de los jugadores. Para cualquier coalición $S \subseteq N$, denotaremos por $x(S)$ al pago total recibido por los miembros de S , i.e., $x(S) = \sum_{i \in S} x_i$.

Definición 2.1.5. Dado un juego $v \in \mathcal{G}_N$, un vector de pago $x \in \mathbb{R}^N$ se dice que es una **impugnación** si satisface las condiciones de:

- **Eficiencia** si asigna la cantidad total obtenida cuando todos cooperan, esto es, $\sum_{i \in N} x_i = v(N)$.
- **Racionalidad individual** si asigna a cada jugador una cantidad superior o igual a su utilidad individual, es decir, $x_i \geq v(i) \forall i \in N$.

Definición 2.1.6. Sea un juego $v \in \mathcal{G}_N$, se dice que una distribución de beneficios o vector de pagos $x \in \mathbb{R}^n$ cumple la condición de **racionalidad coalicional** si $\forall S \subseteq N; x(S) = \sum_{i \in S} x_i \geq v(S)$.

Observación 2.1.2. Un juego que verifique racionalidad coalicional, satisfará también racionalidad individual.

2.2. Valores e Índices de interacción cooperativos

Una de las mayores dificultades en teoría de juegos cooperativos es evaluar la importancia de cada jugador en una coalición. A partir de la Definición 2.1.4, se observa que una función de pagos no considera las utilidades de coaliciones o individuos. Sin embargo, sería razonable que si se hiciese, ya que todo jugador debería recibir una cantidad acorde a su aportación. En este sentido, un vector de pagos puede evaluarse, por ejemplo, según los siguientes criterios:

- **Justicia:** Evalúa cuán bien refleja la utilidad de un jugador su contribución a las coaliciones.
- **Estabilidad:** Evalúa cuáles son los incentivos de los jugadores para permanecer en la coalición en la que se encuentran.

Frente al deseo de una distribución de la cantidad $v(N)$ razonable en términos de justicia y estabilidad, surge el concepto de *valor* o *solución*.

Definición 2.2.1. Una **solución** o **valor** sobre $\mathcal{G}_N$ es una aplicación $\varphi : \mathcal{G}_N \rightarrow 2^{\mathbb{R}^n}$, que a cada juego $v \in \mathcal{G}_N$ le hace corresponder un conjunto de vectores de pago $\varphi(v) \subseteq \mathbb{R}^n$.

Un valor es una correspondencia que asigna a cada juego cooperativo (N, v) un conjunto de vectores de pagos satisfaciendo ciertas condiciones determinadas. Dicho conjunto puede ser vacío, contener un único elemento o varios elementos, según el juego que se considere y las reglas que se establezcan. Por ejemplo, dado un juego $v \in \mathcal{G}_N$ aquella solución cuyas condiciones sean eficiencia y estabilidad racional nos dará el conjunto de *imputaciones* del juego.

El core de un juego cooperativo, desarrollado de manera independiente por Gillies (1953) y Shapley (1952) y, es otra solución conjuntista que satisface las condiciones de eficiencia y racionalidad coalicional. Los vectores de pago de este conjunto son estables y favorecen la formación de la coalición total. Esto es debido a que el pago recibido por cada coalición (o individuo) nunca va a ser menor que la utilidad que obtendría la coalición (o el individuo) por sí sola. De esta manera, ni jugadores ni coaliciones tendrían incentivos (una mayor ganancia) para abandonar la gran coalición.

Por otro lado, las soluciones que seleccionan un único vector de pagos para cualquier juego cooperativo, se conocen como **soluciones puntuales**. Algunos ejemplos podrían ser el **valor de Shapley**, el **núcleo de Schmeidler** o el **valor de Banzhaf**.

Ejemplo 2.2.1 (*Elecciones al Parlamento de Andalucía*).

En las elecciones de 2012 al Parlamento de Andalucía, de un total de 109 escaños, el PP obtuvo 50, el PSOE obtuvo 47 e IU-V obtuvo 12. Para aprobar una ley se necesitan más de la mitad de los votos de la cámara, es decir, una mayoría de 55 votos. Dado que ningún partido tiene por sí solo esta cantidad de escaños, cualquier decisión requiere cooperación entre partidos.

*Podemos modelar esta situación como un **juego simple** en el que los agentes son los partidos y la función característica determina qué coaliciones pueden aprobar leyes. Asignando números a cada partido, $PP \mapsto 1$, $PSOE \mapsto 2$, $IU-V \mapsto 3$, la función característica es la siguiente:*

S	$v(S)$
$\emptyset$	0
$\{i\}, \forall i \in N$	0
$\{1, 2\}$	1
$\{1, 3\}$	1
$\{2, 3\}$	1
$\{1, 2, 3\}$	1

Es decir, para formar una coalición ganadora se necesitan los votos de dos de los tres partidos con representación parlamentaria. Una posible solución para el reparto del poder, podría ser $\varphi(v) = (1, 0, 0)$, que fijémonos se encontraría además dentro del core del juego.

A raíz del ejemplo anterior, vemos que, a pesar de que un vector de pagos perteneciente al core del juego define un pago razonable en términos de eficiencia y racionalidad coalicional, puede llegar a carecer de algunas propiedades deseables para hacerlo más justo. Por ejemplo, en el caso del *Parlamento andaluz* el reparto de poder $(1, 0, 0)$ no es del todo "*justo*" porque el PP no puede aprobar una ley sin la cooperación de los otros partidos. En este sentido nos surge la siguiente pregunta: *¿Qué otras condiciones podrían dar lugar a soluciones más deseables en términos, por ejemplo, de igualdad por roles o recompensa por mérito?*

Definición 2.2.2. Sean (N, v) un juego cooperativo, y $\pi : N \rightarrow N$ una permutación sobre el conjunto de los agentes. El juego $(N, \pi v)$ es aquel que, para cada coalición $S = \{x_1, \dots, x_s\}$, está definido por

$$\pi v(S) = v(\pi(S)) = v(\{\pi(x_1), \dots, \pi(x_s)\})$$

Definición 2.2.3. Sea (N, v) un juego cooperativo, la **contribución marginal** de un jugador $i \in N$ a la coalición $S \subseteq N; i \notin S$ es definida como:

$$MC_i(S) = v(S \cup \{i\}) - v(S)$$

La noción de contribución permite medir cuán valioso es un jugador para las coaliciones con las que potencialmente puede cooperar. A partir de este concepto, podemos definir los distintos tipos de jugadores que puede haber en un juego.

Definición 2.2.4. Sea un juego (N, v) :

- Un conjunto $K \subseteq N$ es **soporte** del juego si $v(S) = v(K \cap S)$ para toda coalición $S \subseteq N$. Además, si un soporte K para el juego v no contiene al jugador $i \in N$, llamamos a i **jugador nulo**, puesto que i no tiene ninguna influencia sobre el valor $v(S)$ de cualquier coalición $S \subseteq N$. Así, todos los jugadores de $N \setminus K$ son nulos.
- Un jugador $i \in N$ es un **títere** en el juego si $v(S \cup \{i\}) = v(S) + v(i) \forall S \subseteq N \setminus \{i\}$. La contribución de este tipo de agentes es independiente del contexto en el que se encuentre.
- Dos jugadores $i, j \in N$ son **indiferentes** en el juego si $v(S \cup \{i\}) = v(S \cup \{j\}) \forall S \subseteq N \setminus \{i, j\}$. Es decir, si estos jugadores tienen roles equivalentes, considerándose intercambiables.

Diremos que una solución $\varphi : \mathcal{G}_N \rightarrow \mathbb{R}^N$:

- Cumple la propiedad de títeres si $\varphi_i(v) = v(i)$ para todo juego v y todo títere $i \in N$.
- Cumple la propiedad de indiferencia si $\varphi_i(v) = \varphi_j(v)$ para todo juego v y toda pareja de jugadores $i, j \in N$ indiferentes.
- Cumple la propiedad de nulidad si $\forall v \in \mathcal{G}_N \forall i \in N$ jugador nulo se tiene que $\varphi_i(v) = 0$.
- Es lineal si $\forall v, w \in \mathcal{G}_N, \alpha \in \mathbb{R}$ y para todo jugador $i \in N$ se cumple que: $\varphi_i(v + \alpha w) = \varphi_i(v) + \alpha \varphi_i(w)$.

Nota. Fijémonos en que, de las nuevas propiedades planteadas, la de *indiferencia* nos llevaría a un reparto de poder en el Ejemplo 2.2.1 más justo en términos de roles equivalentes.

Podemos comprobar que PP, PSOE e IU son jugadores indiferentes; por lo tanto, si nuestra solución φ satisficiera indiferencia, tendríamos que:

$$\begin{cases} \varphi_1(v) = \varphi_2(v) = \varphi_3(v) \geq v(i) = 0 & \forall i \in \{1, 2, 3\} \\ \varphi_1 + \varphi_2 + \varphi_3 = v(123) = 1 \end{cases}$$

Obteniendo así la solución $\varphi(v) = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$, la cual representa mejor la necesidad del apoyo de los diputados de al menos dos de los partidos.

2.2.1. Valor de Shapley

El valor de Shapley fue el primer concepto de solución que asigna a cada juego cooperativo un único vector de pago. Fue introducido y caracterizado de manera axiomática por Shapley (1953). Se trata del concepto de solución sobre el que más se ha escrito e investigado, y es por ello que posteriormente ha sido caracterizado usando diferentes sistemas axiomáticos.

Definición 2.2.5. *El valor de Shapley es una aplicación*

$$\varphi : \mathcal{G}_N \rightarrow \mathbb{R}^N$$

que asigna a cada juego cooperativo v un vector $\varphi(v) = (\varphi_1(v), \dots, \varphi_n(v))$ y cumple las siguientes propiedades:

1. **Eficiencia para soportes.** Si K es soporte del juego v entonces $\sum_{i \in K} \varphi_i(v) = v(K) = v(N)$.
2. **Simetría.** Para todo juego v y toda permutación π de N se cumple $\varphi_i(\pi v) = \varphi_{\pi(i)}(v)$.
3. **Aditividad.** Para todo par de juegos $u, v \in \mathcal{G}_N$ $\varphi(u + v) = \varphi(u) + \varphi(v)$.

Teorema 2.2.1 (Shapley (1953)). *Existe una única función $\phi : \mathcal{G}_N \rightarrow \mathbb{R}^N$ que satisface las propiedades de eficiencia, simetría y aditividad. Esta es la dada por:*

$$\phi_i(v) = \sum_{S \subseteq N; i \in S} \gamma(n, s)(v(S) - v(S \setminus \{i\})), \quad \forall i \in N$$

donde los coeficientes de ponderación son

$$\gamma(n, s) = \frac{(s-1)!(n-s)!}{n!} = \frac{1}{s \binom{n}{s}}$$

La fórmula del valor de Shapley puede ser interpretada como la contribución marginal esperada de cada jugador, al tratarse de una media ponderada del valor añadido que trae un jugador a todas las coaliciones posibles.

Observación 2.2.1. *El valor de Shapley siempre es eficiente. Por lo cual, si el juego es superaditivo, será una imputación.*

Nota. El único concepto de solución para juegos cooperativos que satisface las propiedades de eficiencia, tóteres, nulidad, simetría y linealidad es el valor de Shapley, constituyendo estas propiedades otra caracterización axiomática para este valor.

Por un lado, Shubik (1962) caracterizó el valor de Shapley sustituyendo el axioma de *eficiencia para soportes* por el de *eficiencia* y *jugador nulo*. Y por otro, los axiomas de linealidad y propiedad de tóteres son condiciones más fuertes que las de aditividad y nulidad.

Otra de las **formulaciones** más destacadas del valor de Shapley se basa en el orden de llegada de los jugadores a una coalición, modelado a través de *permutaciones*.

Notación. Sea Π_N el conjunto de todas las permutaciones de N , i.e., de las biyecciones de N en sí mismo. Dada una permutación $\pi \in \Pi_N$, denotamos por $C_\pi(i)$ al conjunto de los predecesores de i en π , luego $C_\pi(i) = \{j \in N | \pi(j) < \pi(i)\}$.

De este modo, la contribución marginal de un agente i respecto a la permutación π en un juego v se define como: $MC_i(C_\pi(i)) = v(C_\pi(i) \cup \{i\}) - v(C_\pi(i))$. Esta cantidad mide el incremento del valor de la coalición formada por los predecesores de i en π cuando este se une.

El valor de Shapley puede definirse como la contribución marginal media, tomando la media sobre todas las posibles permutaciones de N . En otras palabras,

$$\phi_i(v) = \frac{1}{n!} \sum_{\pi \in \Pi_N} MC_i(C_\pi(i)) \quad (2.1)$$

2.2.2. Valor de Banzhaf

Otro concepto de solución *unipuntual* es el **valor de Banzhaf**. Este surgió para juegos simples pero fue extendido a todos los juegos cooperativos por Owen (1972).

Al igual que el valor de Shapley, el valor de Banzhaf se puede interpretar como una media ponderada de las contribuciones marginales de cada jugador a las coaliciones a las que pertenece.

$$\varphi_i^{Banzhaf}(v) := \frac{1}{2^{n-1}} \sum_{S \subseteq N \setminus i} (v(S \cup i) - v(S)) ; \forall v \in \mathcal{G}_N, \forall i \in N$$

Sin embargo, en este caso los coeficientes de ponderación son constantes e iguales a 2^{n-1} , por lo que no dependen del tamaño de la coalición. Feltkamp (1995) demostró que esta solución es la única que satisface los axiomas de simetría, jugador nulo y aditividad.

A pesar de que el valor de Banzhaf no ha sido tan estudiado como el de Shapley, el *índice de Banzhaf* definido a continuación es un conocido índice de poder. Este, a diferencia del valor, si satisface el axioma de eficiencia.

$$f_i^{Banzhaf} := \frac{\sum_{S \subseteq N \setminus i} (v(S \cup i) - v(S))}{\sum_{j=1}^n \sum_{S \subseteq N \setminus j} (v(S \cup j) - v(S))} ; \forall v \in \mathcal{G}_N, \forall i \in N$$

2.2.3. Valores probabilísticos, semivalores

Hasta ahora, hemos analizado los valores de Shapley y Banzhaf desde la perspectiva de los axiomas que caracterizan su comportamiento en juegos cooperativos. Sin embargo, estos valores poseen propiedades que van más allá de las establecidas en los axiomas anteriores. En particular, ambos pueden interpretarse dentro del marco de los *valores probabilísticos*, y más precisamente, como instancias de los *valores cardinales probabilísticos*. Veamos en lo que sigue, qué son este tipo de soluciones, a través de su definición original, aportadas en Weber (1988) y Dubey et al. (1981), respectivamente.

En este último artículo mencionado Dubey y Weber presentaron una definición axiomática del concepto de **semivalor**, que no es más que una particularización de los valores cardinal-probabilísticos, en la que se imponen condiciones de linealidad, simetría y monotonía.

Definición 2.2.6. *Un valor probabilístico φ^p de un jugador $i \in N$ en un juego $v \in \mathcal{G}_N$ es un valor de la forma*

$$\varphi_i^p(v) := \sum_{T \subseteq N \setminus i} p_T^i(N) MC_i(T)$$

donde la familia de coeficientes $\{p_T^i(N)\}_{T \subseteq N \setminus i}$ forma una distribución de probabilidad sobre $2^{N \setminus i}$. Si $i \notin N$, $\varphi_i^p(v) := 0$.

Veamos que $\varphi_i^p(v)$ puede ser interpretado como la esperanza matemática sobre $2^{N \setminus i}$ de la contribución marginal $MC_i(T) = v(T \cup i) - v(T)$ de un jugador i a la coalición $T \subseteq N \setminus i$ con respecto a la distribución de probabilidad $\{p_T^i(N)\}_{T \subseteq N \setminus i}$.

Definición 2.2.7. *Un valor cardinal-probabilístico es un valor probabilístico que cumple adicionalmente que, $\forall i \in N$, los coeficientes $p_T^i(N)$ ($T \subseteq N \setminus i$) dependen solo del cardinal de las*

coaliciones. Es decir, existen n números no negativos $\{p_t(n)\}_{t=0,\dots,n-1}$ verificando

$$\sum_{t=0}^{n-1} \binom{n-1}{t} p_t(n) = 1$$

tal que para cualquier $i \in N$ y cualquier $T \subseteq N \setminus i$, tenemos que $p_T^i(N) = p_t(n)$.

Fijémonos en que, por un lado, el *valor de Banzhaf* surge de la hipótesis de que un jugador se une a una coalición de forma equiprobable, es decir, la probabilidad de que un jugador $i \in N$ se una a una coalición $T \subseteq N \setminus i$ es $p_T^i = \frac{1}{2^{n-1}} \forall T \subseteq N \setminus i$. Mientras que, el *valor de Shapley*, surge de la suposición de que es igualmente probable unirse a una coalición de cualquier tamaño t ($0 \leq t \leq n-1$), y que todas las coaliciones de tamaño t son equiprobables.

2.2.4. Índices de interacción

Para introducir el concepto de **interacción** entre dos agentes de un juego cooperativo plantearemos un ejemplo extraído de Lange (2007).

Ejemplo 2.2.1. Sea una capacidad v sobre el conjunto finito de agentes $N = \{1, 2\}$, tal que $v(N) = 1$ y $v(\emptyset) = 0$. El valor de Shapley del juego viene dado por:

$$\begin{cases} \phi_1^S(v) = \frac{1}{2}v(1) + \frac{1}{2}(1 - v(2)), \\ \phi_2^S(v) = \frac{1}{2}v(2) + \frac{1}{2}(1 - v(1)) \end{cases}$$

Supongamos que cada uno de los dos jugadores tiene la misma *importancia*, es decir, $\phi_1^S(v) = \phi_2^S(v)$. De esta igualdad, se deduce que $v(1) = v(2)$. Es claro que, dependiendo de la utilidad asignada a cada uno de los agentes, su **interacción** puede ser beneficiosa o no. Consideremos los casos extremos:

- Si $v(1) = v(2) = 0$, la interacción entre los jugadores es **complementaria**, ya que, aunque individualmente no aportan valor, su cooperación les permite alcanzar una retribución de $\frac{1}{2}$ a cada uno.
- Si $v(1) = v(2) = 1$, los jugadores son **redundantes**, puesto que individualmente cualquiera de ellos puede generar el valor total del juego. Su cooperación en este caso no añade valor adicional, y podríamos considerar que la presencia de los dos no sería, en cierta medida indispensable.

En la situación intermedia $v(1) = v(2) = \frac{1}{2}$, la función de utilidad es *aditiva*, lo cual señala cierta independencia entre los agentes.

El ejemplo anterior nos ayuda a darnos cuenta de que, en general, los jugadores tienen cierto interés en formar coaliciones y que pueden darse tres tipos de situación. Sea N un conjunto de jugadores finito tal que $i, j \in N$, y $v \in \mathcal{G}_N$ una función característica sobre N .

- $v(ij) > v(i) + v(j)$: Los jugadores i, j interactúan positivamente.
- $v(ij) < v(i) + v(j)$: Los jugadores i, j interactúan negativamente.
- $v(ij) = v(i) + v(j)$: Los jugadores i, j tienen roles independientes en el juego.

Sería natural pensar que, en el caso de que existiese un coeficiente que midiese la interacción entre los jugadores i y j , este debería depender de la diferencia $v(ij) - [v(i) + v(j)]$. No obstante, para que este análisis fuera global, también sería necesario considerar las contribuciones conjuntas de estos jugadores a otras coaliciones.

Definición 2.2.8. Dada una función característica $v : 2^N \rightarrow \mathbb{R}$ y dos coaliciones finitas $S, T \subseteq N$ tales que $S \cap T = \emptyset$, la S -derivada de v sobre T , $\Delta_S(v(T))$, se define de manera recursiva como sigue:

$$\begin{aligned}\Delta_i v(T) &= v(T \cup i) - v(T) \quad \forall i \in N \\ \Delta_S(v(T)) &= \Delta_i[\Delta_{S \setminus i} v(T)] = \sum_{L \subseteq S} (-1)^{s-l} v(T \cup L) \quad \forall i \in S\end{aligned}$$

Donde esta última igualdad puede ser obtenida por inducción sobre s .

Nosotros nos vamos a centrar en el caso especial de subconjuntos de tamaño dos, $S = \{i, j\}$. La expresión

$$\Delta_{\{i,j\}} v(T) = v(T \cup ij) - v(T \cup i) - v(T \cup j) + v(T)$$

puede verse como la diferencia entre las contribuciones marginales $\Delta_j v(T \cup i) = v(T \cup ij) - v(T \cup i)$ y $\Delta_i v(T) = v(T \cup j) - v(T)$. Es por este motivo que Grabisch llamó a esta diferencia la interacción marginal entre i y j en presencia de la coalición T .

El concepto de **índice de interacción** fue introducido por Owen (1972) como una extensión del concepto de *valor*. Este definió el índice de interacción entre $i, j \in N$ en un juego $v \in \mathcal{G}_N$ como sigue:

$$I_{i,j}(v) := \sum_{T \subseteq N \setminus \{i,j\}} \frac{1}{n-1} \binom{n-2}{t}^{-1} \Delta_{\{i,j\}} v(T)$$

Tratándose de la media ponderada de las interacciones marginales entre los jugadores i, j en presencia de T , una coalición que no contiene a ninguno de estos agentes.

Fue gracias a Grabisch (1997) y Roubens (1996), quiénes presentaron los índices de interacción de Shapley y Banzhaf, que este concepto comenzó a extenderse a coaliciones con más de dos jugadores.

$$\begin{aligned}I_S(v) &= \sum_{T \subseteq N \setminus S} \frac{1}{(n-s+1)} \binom{n-s}{t}^{-1} \Delta_S v(T) \\ I_S^{\text{Banzhaf}} &:= \sum_{T \subseteq N \setminus S} \frac{1}{2^{n-1}} \Delta_S v(T)\end{aligned}$$

Fijémonos en que $I_i(v) = \varphi_i(v) \quad \forall i \in N, v \in \mathcal{G}_N$.

Definición 2.2.9. Sea $v \in \mathcal{G}_N$, el **índice de interacción** de v es una función $I(v) : 2^N \rightarrow \mathbb{R}$ que asigna a cada coalición $T \subseteq N$ su grado de interacción.

De forma similar a la definición de valores probabilísticos, Fujimoto et al. (2006) introdujo los índices de interacción probabilísticos.

Definición 2.2.10. Sea $S \subseteq N$ una coalición y $v \in \mathcal{G}_N$ un juego, el **índice de interacción probabilístico** de la coalición es de la forma:

$$I_S^p(v) := \sum_{T \subseteq N \setminus S} p_T^S(N) \Delta_S v(T)$$

donde para cada $S \subseteq N$, la familia de coeficientes $\{p_T^S(N)\}_{T \subseteq N \setminus S}$ forma una distribución de probabilidad sobre $2^{N \setminus S}$. Si $S \not\subseteq N$, $I_S^p(v) := 0$.

Definición 2.2.11. Un *índice de interacción cardinal-probabilístico* es un índice de interacción probabilístico, el cual cumple adicionalmente que para cualquier $S \subseteq N$, la familia de coeficientes $\{p_T^S(N)\}$ con $T \subseteq N \setminus S$ depende únicamente de las cardinales de las coaliciones S, T y N . Es decir, para cualquier $s \in \{0, \dots, n\}$, existe una familia de números no negativos $\{p_t^s(n)\}_{t=0, \dots, n-s}$ cumpliendo que

$$\sum_{t=0}^{n-s} \binom{n-s}{t} p_t^s(n) = 1$$

tal que $\forall S \subseteq N, \forall T \subseteq N : p_T^S(N) = p_t^s(n)$.

2.3. Teoría de grafos

La teoría de grafos surgió en el S.XVII a raíz del trabajo de Leonhard Euler sobre el *problema de los puentes de Königsberg*. Desde entonces, los grafos se emplean en múltiples campos. En este trabajo los grafos serán la estructura que dará soporte a las redes de agentes racionales, sobre las que se definirá un juego cooperativo.

Definición 2.3.1. Un **grafo** G es un par ordenado $G = (V, E)$, donde V es un conjunto no vacío de elementos denominados *vértices* o *nodos* y $E \subseteq V \times V$ es un conjunto de pares $\{u, v\}$ no ordenados llamados *aristas*.

Definición 2.3.2. Sea $u \in V$ un nodo, definimos el **grado** de u , representado como $\deg(u)$, como número de vértices a los que u está conectado a través de una arista: $\deg(u) := |\{v \in V \mid (u, v) \in E\}|$. Además, si d una métrica o distancia definida sobre V y $k \in \mathbb{N}$, el **grado k -vecinal** de u se define como: $\deg_k(u) := |\{v \in V \mid d(u, v) \leq k\}|$.

Definición 2.3.3. Sean $i, j \in V$ dos vértices y $d(i, j)$ la distancia entre estos, definimos la distancia de un subconjunto de nodos $S \subseteq V$ a otro vértice $i \in V$:

$$d(S, i) := \min_{j \in S} d(i, j)$$

Definición 2.3.4. Sean $u \in V$, $k \in \mathbb{N}$ y d una métrica sobre V , definimos la **k -vecindad** del nodo u , denotada por $N_k(u)$, como el conjunto de vértices del grafo cuya distancia a u es menor o igual a k . Es decir,

$$N_k(u) := \{v \in V \setminus u \mid d(u, v) \leq k\}$$

extendiéndolo a subconjuntos de nodos $S \subseteq V$ como $N_k(S) = \bigcup_{v \in S} N_k(v) \setminus S$.

Nos referiremos a este conjunto también como la **k -esfera de influencia**, tanto para un nodo individual como para un conjunto de ellos.

Capítulo 3

Índices de interacción en redes

En este capítulo exploraremos algunos métodos juego-teóricos para modelar la similitud de nodos en una red. A partir de ahora, consideraremos juegos cooperativos definidos mediante funciones características sobre el conjunto de vértices de una red.

En este contexto, los vértices del grafo actuarán como jugadores, mientras que las aristas no indicarán directamente relaciones cooperativas, sino que establecerán distancias entre ellos. Así, las redes con las que trabajaremos serán no dirigidas ni ponderadas.

La similitud entre dos jugadores será definida en términos de su índice de interacción. El cálculo del índice de interacción en un juego cooperativo arbitrario es un problema costoso a nivel de complejidad, ya que implica computar la contribución marginal de un par de nodos $i, j \in V$ a toda coalición $C \subseteq V$. Dado que el número de coaliciones es $2^{|V|}$, el costo computacional crece de manera exponencial con respecto al número de jugadores.

A pesar de este desafío computacional, es posible derivar expresiones que reducen significativamente la complejidad. En particular, presentaremos a continuación dos índices aplicados a redes cuya computación puede realizarse en tiempo polinomial. Además, se proporcionarán los algoritmos implicados en su cálculo, los cuales han sido programados y se encuentran en el código del Anexo A.

3.1. Planteamiento del problema

La mayoría de las medidas de similitud topológica entre dos nodos en una red se basan en la estructura de sus vecindades. En este capítulo, el objetivo principal es definir la interacción en redes como un mecanismo para obtener una medida de similitud. Para ello, se emplearán medidas locales, teniendo en cuenta únicamente vecindades de radio k o k -vecindades, con $k \in \mathbb{N}$.

Definición 3.1.1 (Tarkowski et al. (2019)). *Sea una red $G = (V, E)$ y $S \subseteq V$ una coalición, definimos la función característica **f-medida de influencia** de la siguiente manera:*

$$v_f(S) = \sum_{u \in V \setminus S} f(d(S, u)) \quad (3.1)$$

donde $d : 2^V \times 2^V \rightarrow \mathbb{R}$ es una función de distancia entre subconjuntos de vértices, definida en términos de caminos mínimos, y $f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ es una función de proximidad.

Nota. En este trabajo la función f será una función **inversamente proporcional** a alguna de las potencias de d . Luego será una función decreciente, que garantizará además $f(0) = 0$, como

condición adicional. Teniendo esto en cuenta, la función **f-medida de influencia** se puede redefinir como sigue

$$v_f(S) = \sum_{u \in V} f(d(S, u)) \quad (3.2)$$

Cada función de proximidad f define distintas funciones características y, por ende, diferentes índices de interacción. No obstante, todas las **funciones características** v_f generadas serán **débilmente subaditivas**. Para comprobarlo estudiemos el signo de la siguiente expresión $f(d(S \cup i, u)) - f(d(S, u)) - f(d(i, u))$ para cada caso. Sean $u, i \in V$,

- En el caso en que $u \neq i, S \subseteq V \setminus \{u, i\}$:
 - Si $d(i, u) < d(S, u) : f(d(S, u)) < f(d(i, u))$. Luego en este caso $f(d(S \cup i, u)) = f(d(i, u)) \leq f(d(i, u)) + f(d(S, u))$.
 - Si, por el contrario, $d(i, u) \geq d(S, u)$, llegaríamos a que $f(d(S \cup i, u)) = f(d(S, u))$.
- Si $u = i$: $f(d(u, i)) = f(d(S \cup i, u)) = 0$.
- Si $i \in S$: $f(d(S \cup i, u)) = f(d(S, u))$.
- Si $u \in S$: $f(d(S \cup i, u)) = f(d(S, u)) = 0$.

En cualquiera de los escenarios tenemos que $f(d(S \cup i, u)) \leq f(d(S, u)) + f(d(i, u))$. Entonces, si se cumple para todo vértice $u \in V \setminus S$, el sumatorio sobre todos los vértices también lo cumplirá. De manera que $v_f(S \cup i) \leq v_f(S) + v_f(i)$, es decir, v_f es un juego subaditivo débil.

Definición 3.1.2. Decimos que la **sinergia** entre dos jugadores $i, j \in V$ en presencia de una coalición $C \subseteq V$, es el valor añadido que aporta la cooperación de estos dos a la coalición en comparación con el que alcanzan de manera individual. Esta se define entonces como:

$$S_v(C, i, j) = MC_{\{i, j\}}(C) - MC_i(C) - MC_j(C) = \Delta_{\{i, j\}}v(C) \quad (3.3)$$

Observación 3.1.1. La sinergia será cero en el caso de que alguno o ambos jugadores pertenezcan a la coalición. En particular, si $i \in C$, entonces:

$$S_v(C, i, j) = MC_{\{i, j\}}(C) - MC_j(C) = v(C \cup j) - v(C) - (v(C \cup j) - v(C)) = 0$$

Notación. Sean $i, j \in V$ nodos del grafo y $C \subseteq V$ una coalición, expresamos por $C^t \subseteq V$ el conjunto de coaliciones de tamaño t . Es más,

$$C^t(V \setminus \{i, j\}) = \{C \subseteq V \setminus \{i, j\} \mid |C| = t\}$$

representa el conjunto de coaliciones de tamaño t que no contienen a los vértices i, j del grafo.

Definición 3.1.3. El **índice de interacción relativo al semivalor** v sobre V se representa como

$$I_{i, j}^{Semi}(v) = \sum_{0 \leq t \leq |V|-2} \beta(t) \mathbb{E}[S_v(C^t, i, j)] \quad (3.4)$$

donde $\beta : \{0, 1, \dots, |V| - 2\} \rightarrow [0, 1]$ es una función de probabilidad discreta y

$$\mathbb{E}[S_v(C^t, i, j)] = \sum_{C \in C^t(V \setminus \{i, j\})} \frac{S_v(C, i, j)}{\binom{|V|-2}{t}}$$

es la esperanza de la sinergia entre los jugadores i, j en presencia de una coalición de tamaño t .

Observación 3.1.2. *Los dos índices de interacción cardinales probabilísticos introducidos previamente toman las siguientes funciones de distribución:*

- $\beta(t) = \frac{1}{|V|-1}$, para el índice de interacción de **Shapley**.
- $\beta(t) = \frac{\binom{|V|-2}{t}}{2^{|V|-2}}$, para el índice de interacción de **Banzhaf**.

Observación 3.1.3. *En nuestra situación, dado que v_f es subaditiva, el valor de la sinergia y del índice de interacción será no positivo:*

$$S_{v_f}(C, i, j) = v_f(C \cup \{i, j\}) - v_f(C \cup i) - v_f(C \cup j) + v_f(C) \leq 0$$

Notemos que estos conceptos ya fueron presentados en la Sección 2.2.4. Estamos únicamente introduciendo la notación que usaremos de ahora en adelante. De hecho, vamos a tener que recuperar la definición (2.1) del valor de Shapley por permutaciones, de la cual deriva el siguiente índice de interacción:

$$I_{i,j}^{Shapley}(v) = \sum_{\pi \in \Pi(V^{i \wedge j})} \frac{S_v(C_\pi(\{i, j\}), i, j)}{(n-1)!} \quad (3.5)$$

donde $V^{i \wedge j} = V \setminus \{i, j\} \cup \{\{i, j\}\}$.

Veamos por qué la función (3.1) recibe el nombre de **f-medida de influencia**. Existen gran variedad de definiciones de juegos de influencia en redes; en nuestro caso, la definición será una extensión de la presentada en Szczepanski et al. (2015).

Definición 3.1.4. *Un k -juego de influencia sobre un grafo $G = (V, E)$ es una tupla (V, v_I^k) donde $v_I^k : 2^V \rightarrow \mathbb{R}$ es una función de influencia tal que:*

$$v_I^k(C) = \sum_{v \in N_k(C)} f(v, C)$$

siendo $f : V \times 2^V \rightarrow \mathbb{R}$ una aplicación que asigna a cada nodo de la esfera de influencia $v \mapsto f(v, C)$, una contribución a la coalición.

Una manera de definir la similitud entre dos vértices de la red podría ser a través del solapamiento de sus k -esferas de influencia. Esta noción de similitud se asemeja a considerar la intersección de sus k -vecindades. Así, sean dos vértices del grafo: cuanto mayor sea el solapamiento de sus k -esferas de influencia, más similares resultarán.

Este concepto de similitud dará lugar a interacciones negativas, lo cual puede parecer a priori contraintuitivo. Presentaremos a continuación un ejemplo en el que se muestra un juego cooperativo donde la similitud y las interacciones positivas no van de la mano.

Ejemplo 3.1.1 (*El mercado de los guantes*).

Sea $N = \{1, 2, 3, 4\}$ el conjunto de agentes de nuestro juego. Cada uno de ellos posee un tipo de guante, pudiendo ser este para su mano izquierda o derecha. En particular los jugadores 1 y 2 tienen un guante para la mano derecha, mientras que 3 y 4 tienen uno correspondiente a la izquierda.

Definimos la función característica v a través de las siguientes consideraciones:

- El valor de un solo guante es cero.
- El valor de cualquier par de guantes de manos diferentes es de 1 euro.

Luego $v : 2^N \rightarrow \mathbb{R}$ queda definida de manera explícita en 3.1.

S	$v(S)$
$\emptyset$	0
$\{i\}, \forall i \in N$	0
$\{1, 2\}$	0
$\{3, 4\}$	0
$\{i, j\}, \forall \{i, j\} \neq \{1, 2\}, \{3, 4\}$	1
$\{i, j, k\}, \forall \{i, j, k\} \subseteq N$	1
$\{1, 2, 3, 4\}$	2

Cuadro 3.1: Valores de la función $v(S)$ para distintos subconjuntos S .

En este caso $1, 2 \in N$ son dos jugadores similares, al poseer la misma característica, un guante en la mano derecha. Los posibles tamaños de las coaliciones a las cuales pueden contribuir son:

- $t = 0$: $C = \emptyset$. Obteniendo así, $S_v(C, 1, 2) = 0$
- $t = 1$: $C \in \{\{3\}, \{4\}\}$. En ambos casos $S_v(C, 1, 2) = -1$
- $t = 2$: $C = \{3, 4\}$, $S_v(C, 1, 2) = 0$

Por lo tanto el índice de interacción de Shapley es: $I_{1,2}^{Shapley}(v) = -\frac{1}{3}$.

El ejemplo anterior forma parte de un tipo de juego cooperativo llamado *juegos de asignación*. Estos juegos modelizan situaciones en las que hay dos tipos de agentes diferentes, como es el caso de una red de compradores y vendedores. En estas situaciones, dos jugadores con roles **complementarios** generan un valor **adicional** a través de la cooperación, mientras que aquellos más similares no. Esto sucede porque en estos escenarios la similitud conlleva **competencia**.

En las siguientes secciones daremos definiciones particulares de la **f-medida de influencia** definida en (3.1) y veremos que esta situación de conflicto entre la similitud de dos nodos y el valor obtenido a través de su cooperación se da en todas ellas.

Observación 3.1.4. Es importante destacar que para toda **f-medida de influencia** v_f , es necesario conocer las distancias mínimas entre todos los pares de nodos del grafo. Puesto que en este trabajo solo se consideran grafos no ponderados, utilizaremos el conocido algoritmo *Breadth-First search (BFS)*, cuya implementación se encuentra en el Anexo A. Este algoritmo tiene una complejidad temporal de $O(|V|^2 + |V||E|)$, siendo $|V|, |E|$ el número de vértices y aristas del grafo respectivamente.

3.2. Índice de interacción de Szczepáński

Sea $k \in \mathbb{N}$ un número natural cualquiera,

$$f(d) = \begin{cases} 1 & \text{si } 0 < d \leq k, \\ 0 & \text{en otro caso} \end{cases}$$

da lugar a la **función de influencia de k-pasos** expuesta en Szczepanski et al. (2015):

$$v_k(S) = |\{v \in V \setminus S \mid d(S, v) \leq k\}| \quad (3.6)$$

En esta y la siguiente sección, nos centraremos en computar el valor del término $S_v(C, i, j)$ para diferentes funciones características. Antes de enunciar los teoremas correspondientes a los índices

de interacción introducidos en Szczepanski et al. (2015), deduciremos un desarrollo de la expresión $S_{v_k}(C, i, j)$ que, basado en el razonamiento de dicho artículo, aclarará conceptos clave utilizados en estos teoremas.

Proposición 3.2.1. *Sea v_k la función de influencia de k -pasos, $C \subseteq V \setminus \{i, j\}$ una coalición cualquiera que no contiene a los nodos i, j ,*

$$S_{v_k}(C, i, j) = |\{u \in (N_k(i) \cap N_k(j)) \cup (N_k(i) \cap \{j\}) \cup (N_k(j) \cap \{i\}) \mid u \notin C, u \notin N_k(C)\}|$$

Demostración. Sea $C \subseteq V \setminus \{i, j\}$ una coalición cualquiera, dividamos el problema del cálculo de $S_{v_k}(C, i, j)$ en el estudio de las contribuciones marginales $MC_i(C)$, $MC_j(C)$, $MC_{\{i,j\}}(C)$. Decimos que un nodo $i \in V$:

- Contribuye positivamente a la coalición C si $\exists u \in N_k(i) : u \notin C, u \notin N_k(C)$. Esto sucede porque i ampliaría la esfera de influencia de la coalición con la incorporación de u como nuevo vecino de la k -vecindad. Por lo tanto, $MC_i^+(C) = |\{u \in N_k(i) \mid u \notin C, u \notin N_k(C)\}|$.
- Contribuye negativamente si $i \in N_k(C)$, puesto que al unirse i a la coalición ya no formaría parte de su k -esfera de influencia. Así,

$$MC_i^-(C) = \begin{cases} 1 & \text{si } i \in C \\ 0 & \text{en caso contrario} \end{cases} \quad (3.7)$$

Analicemos ahora cuáles pueden ser las contribuciones marginales del par de nodos, siendo las de j análogas a las de i . El conjunto $\{i, j\} \subseteq V$ contribuye:

- Positivamente si $\exists u \in (N_k(i) \setminus j) \cup (N_k(j) \setminus i) : u \notin C, u \notin N_k(C)$. Por ello, $MC_{\{i,j\}}^+(C) = |\{u \in (N_k(i) \cup N_k(j)) \setminus \{i, j\} \mid u \notin C, u \notin N_k(C)\}|$.
- Negativamente a la coalición C si bien alguno de los nodos pertenecía a su esfera de influencia antes de incorporarse. Es decir, $MC_{\{i,j\}}^-(C) = MC_i^-(C) - MC_j^+(C)$.

De este modo, llegamos a la siguiente reducción:

$$\begin{aligned} S(C, i, j) &= MC_{\{i,j\}}^+(C) - MC_i^+(C) - MC_j^+(C) - (MC_{\{i,j\}}^-(C) - MC_i^-(C) - MC_j^-(C)) \\ &= |\{u \in (N_k(i) \cap N_k(j)) \cup (N_k(i) \cap \{j\}) \cup (N_k(j) \cap \{i\}) \mid u \notin C, u \notin N_k(C)\}| \end{aligned} \quad (3.8)$$

□

Primero presentaremos el índice de interacción derivado de (3.4). En este caso, para cada $t \in \{0, \dots, |V| - 2\}$, tomando el número de posibles coaliciones de este tamaño ($C^t(V \setminus \{i, j\})$) y una función $\beta(t) : \{1, \dots, |V| - 2\} \rightarrow \mathbb{R}$ de distribución de probabilidad discreta, obtenemos la expresión del índice.

Teorema 3.2.1 (Szczepanski et al. (2015)). *Sea $G = (V, E)$ un grafo, $i, j \in V$ un par de nodos, el índice de interacción relativo al semivalor de influencia de k -pasos puede ser computado en tiempo polinomial sobre $|V|$.*

Demostración. Sean $i, j \in V$ dos nodos y $C^t \subseteq V \setminus \{i, j\}$, introduciremos dos variables aleatorias de Bernouilli $B_{t,i,u}^+$, $B_{t,i}^-$, las cuales indicarán si el nodo i contribuye positiva o negativamente a la coalición C^t cuando se une a esta, tal y como se vió en la Proposición 3.2.1.

- El nodo i contribuye positivamente. En este caso, $\mathbb{E}[B_{t,i,u}^+] = P[(N_k(u) \cup \{u\}) \cap C^t = \emptyset]$.
- El nodo i contribuye negativamente. La esperanza de que esto ocurra es $\mathbb{E}[B_{t,i}^-] = P[N_k(i) \cap C^t \neq \emptyset]$.

Fijémonos en que, para determinar estas esperanzas, hemos de tener en cuenta también al nodo $j \in V$.

$$\mathbb{E}[B_{t,i,u}^+] = \begin{cases} \frac{\binom{|V|-1-\deg_k(u)}{t}}{\binom{|V|-2}{t}} & \text{si } u = j, \text{ o } j \in N_k(u) \\ \frac{\binom{|V|-2-\deg_k(u)}{t}}{\binom{|V|-2}{t}} & \text{en caso contrario} \end{cases} \quad (3.9)$$

Por otro lado, dado que $P[N_k(i) \cap C^t \neq \emptyset] = 1 - P[N_k(i) \cap C^t = \emptyset]$, tenemos

$$\mathbb{E}[B_{t,i}^-] = \begin{cases} 1 - \frac{\binom{|V|-1-\deg_k(i)}{t}}{\binom{|V|-2}{t}} & \text{si } i \in N_k(j) \\ 1 - \frac{\binom{|V|-2-\deg_k(i)}{t}}{\binom{|V|-2}{t}} & \text{en caso contrario} \end{cases} \quad (3.10)$$

La esperanza de la contribución marginal de $i \in V$ a una coalición de tamaño t se expresará como: $\mathbb{E}[MC_i(C^t)] = \sum_{u \in N_k(i)} \mathbb{E}[B_{t,i,u}^+] - \mathbb{E}[B_{t,i}^-]$. Para $j \in V$ expresión es análoga, por lo que solo falta computar $\mathbb{E}[MC_{\{i,j\}}(C^t)]$. Seguimos el mismo razonamiento llegando finalmente a que:

$$\mathbb{E}[S(C^t, i, j)] = \sum_{\substack{u \in (N_k(i) \cap N_k(j)) \cup \\ (\{i\} \cap N_k(j)) \cup (\{j\} \cap N_k(i))}} \frac{\binom{|V|-1-\deg_k(u)}{t}}{\binom{|V|-2}{t}}$$

Una vez obtenida la expresión para el cálculo de la sinergia, se ha de iterar sobre los posibles tamaños $t \in \{0, \dots, |V| - 2\}$ de coaliciones a las que los nodos i y j pueden contribuir. Como esta iteración abarca $O(|V|)$ valores de t posibles, el cálculo de interacción para un par de nodos se realiza en $O(|V|^2)$. $\square$

Algorithm 1: Índice de interacción de k-grado de un semivalor

Input: Grafo $\mathbf{G} = (\mathbf{V}, \mathbf{E})$: red de jugadores en forma de grafo, $i, j \in V$

β : función de distribución de probabilidad

Output: $\mathbf{I}_{i,j}^{\text{Semi}}$: índice de interacción de k-grado de un semivalor

$I^{\text{Semi}}(i, j) \leftarrow 0$;

for $t \leftarrow 0$ **to** $|V| - 2$ **do**

$S_t \leftarrow 0$;

for $u \in (N_k(i) \cap N_k(j))$ **do**

$S_t \leftarrow S_t - \frac{\binom{|V|-1-\deg_k(u)}{t}}{\binom{|V|-2}{t}}$;

if $j \in N_k(i)$ **then**

$S_t \leftarrow S_t - \frac{\binom{|V|-1-\deg_k(j)}{t}}{\binom{|V|-2}{t}} - \frac{\binom{|V|-1-\deg_k(i)}{t}}{\binom{|V|-2}{t}}$;

$I_{i,j}^{\text{Semi}} \leftarrow I_{i,j}^{\text{Semi}} + \frac{\beta(t)}{\binom{|V|-2}{t}} S_t$;

La siguiente demostración es de elaboración propia, a pesar de estar inspirada en los teoremas de los dos artículos principales (Tarkowski et al. 2019) (Szczepanski et al. 2015). La expresión resultante de esta proposición no es una particularización del índice general anterior, sino que deriva de la expresión del índice de interacción de Shapley por permutaciones.

Teorema 3.2.2. Sea v_k la función **de influencia de k -pasos**, el índice de interacción de Shapley derivado de (3.5) se expresa como sigue:

$$I_{i,j}^{Shapley} = \begin{cases} \sum_{u \in (N_k(i) \cap N_k(j))} -\frac{1}{deg_k(u)-1} & \text{si } j \notin N_k(i) \\ \sum_{u \in (N_k(i) \cap N_k(j))} \left(-\frac{1}{deg_k(u)-1}\right) - \frac{1}{deg_k(i)-1} - \frac{1}{deg_k(j)-1} & \text{en caso contrario} \end{cases} \quad (3.11)$$

Demostración. Retomamos la expresión 3.5 del índice de Shapley por *permutaciones*:

$$I_{i,j}^{Shapley}(v) = \sum_{\pi \in \Pi(V^{i \wedge j})} \frac{S_v(C_\pi(\{i, j\}), i, j)}{(|V| - 1)!}$$

Para cada coalición C_π , los nodos que pueden contribuir a la sinergia $S(C_\pi(\{i, j\}), i, j)$ son aquellos que se encuentran a distancia mayor que k de $C_\pi(\{i, j\})$.

Seguindo la expresión de la sinergia (3.3), cada vértice

$$u \in (N_k(i) \cap N_k(j)) \cup (N_k(i) \cap \{j\}) \cup (N_k(j) \cap \{i\})$$

aparecerá en la expresión final del índice tantas veces como coaliciones a distancia mayor que k de u existan.

Por consiguiente, el problema se reduce a calcular, para cada nodo $u \in V$ el cardinal del siguiente conjunto: $C_u := \{\pi \in \Pi(V^{i \wedge j}) \mid d(C_\pi(\{i, j\}), u) > k\}$. De esta manera podemos reescribir el índice como sigue:

$$I_{i,j}^{Shapley}(v) = \frac{1}{(|V| - 1)!} \sum_{\substack{u \in (N_k(i) \cap N_k(j)) \cup \\ (\{i\} \cap N_k(j)) \cup (\{j\} \cap N_k(i))}} |C_u|$$

Dado que las permutaciones establecen un orden de llegada a la coalición, lo anterior es equivalente a contar aquellas permutaciones $\pi : V^{i \wedge j} \rightarrow V^{i \wedge j}$ en las que el conjunto de predecesores de $\{i, j\}$, que forman $C_\pi(\{i, j\})$, se encuentra a una distancia mayor que k de u .

Es importante notar que una coalición no posee un orden intrínseco, mientras que una permutación sí lo impone. Por lo tanto, para cada posible conjunto de jugadores que precede a $\{i, j\}$ y cumple la restricción de distancia, debemos considerar todas las posibles formas en que estos jugadores pueden ordenarse, así como los órdenes de los agentes que suceden a $\{i, j\}$. Esto se debe a que cada orden específico define una permutación π distinta.

Tenemos así que el número de permutaciones que cumplen esto es :

$$\binom{|V| - 1}{deg_k(u)} (deg_k(u) - 2)! (|V| - 1 - (deg_k(u) - 1))! = \frac{(|V| - 1)!}{deg_k(u) - 1} \quad (3.12)$$

Llegando así a la siguiente expresión:

$$I_{i,j}^{Shapley}(v) = \sum_{\substack{u \in (N_k(i) \cap N_k(j)) \cup \\ (\{i\} \cap N_k(j)) \cup (\{j\} \cap N_k(i))}} \frac{1}{deg_k(u) - 1}$$

□

Algorithm 2: Índice de interacción de k-grado de Shapley

Input: Grafo $\mathbf{G} = (\mathbf{V}, \mathbf{E})$: red de jugadores en forma de grafo, $i, j \in V$

$\forall u \in V : N_k(u)$ -conjunto de k-vecinos

Output: $I_{i,j}^{\text{Shapley}}$ índice de interacción de k-grado de Shapley

$I_{i,j}^{\text{Shapley}} \leftarrow 0$;

for $u \in (N_k(i) \cap N_k(j))$ **do**

$I_{i,j}^{\text{Shapley}} \leftarrow I_{i,j}^S - \frac{1}{\deg_k(u)-1}$;

if $j \in N_k(i)$ **then**

$I_{i,j}^{\text{Shapley}} \leftarrow I_{i,j}^{\text{Shapley}} - \frac{1}{\deg_k(j)-1} - \frac{1}{\deg_k(i)-1}$;

Notemos que en este caso, al ser necesario iterar sobre los tamaños de coalición $t \in \{0, \dots, |V| - 1\}$, el cálculo del índice de interacción entre dos nodos puede realizarse en $O(|V|)$.

Por lo tanto, en una red $G = (V, E)$ una red, la complejidad temporal del cálculo del índice de interacción para todos los pares de nodos es $O(|V|^4)$ en el caso del índice basado en **semivalores**, $I_{i,j}^{\text{Semi}}(v_k)$. Esta complejidad se reduce a $O(|V|^3)$ en el caso del índice de interacción de **k-grado de Shapley**, $I_{i,j}^{\text{Shapley}}(v_k)$.

3.3. Índice de interacción de Tarkowski

Sea $k \in \mathbb{N}$ un número natural cualquiera,

$$f(d) = \begin{cases} \frac{1}{d^2} & \text{si } 0 < d \leq k, \\ 0 & \text{en otro caso} \end{cases} \quad (3.13)$$

nos proporciona la **función de influencia de k-pasos amortiguada** de Tarkowski et al. (2019):

$$v^T(C) = \sum_{u \in V} f(d(C, u)) = \sum_{u \in V \setminus C} \frac{1}{d(C, u)^2} \quad \forall C \subseteq V \quad (3.14)$$

Esta función característica preserva la propiedad de que los nodos fuera de la k-vecindad no impacten a la medida e incorpora, además, un principio de *decrecimiento de influencia*, según el cual, cuanto más cercano está un nodo dentro de la k-vecindad, mayor será su impacto en la similitud. En consecuencia, los nodos más alejados tendrán una contribución menor.

Notación. Sean $u, i, j \in V$ nodos del grafo y $C \subseteq V$ una coalición, en lo que sigue:

- Cada uno de los sumandos de (3.14): $v_u(C) = f(d(C, u)) = \begin{cases} \frac{1}{d(C, u)^2} & \text{si } 0 < d(C, u) \leq k \\ 0 & \text{en caso contrario} \end{cases}$
 $\forall C \subseteq V$ es también una función de utilidad restringida a un nodo $u \in V$
- Denotaremos por $S_{v_u}(C, i, j)$ y $MC_{\{i,j\}}^{v_u}(C)$ la sinergia y la contribución marginal de $i, j \in V$ en presencia de $C \subseteq V$ relativas a v_u .
- $\text{Nodos}_{\sim_d}(u) = \{w \in V \mid w \sim_d u\}$ es un conjunto definido para las relaciones de órdenes parciales $<, >, \leq, \geq$, que alberga todos aquellos nodos $w \in V$ tales que $d(w, u) \sim d$.
- $\text{distancias}(u) = \{d \mid \exists w \in N_k(u); d = d(u, w)\}$ es el conjunto de distancias a las que se encuentra al menos un nodo del grafo de u dentro de su k -vecindad.

- Representamos por V_k el número medio de nodos a una distancia menor o igual a k de un vértice cualquiera del grafo.

Sea V el conjunto de agentes de nuestra red, $i, j \in V$, y $C \in C(V \setminus \{i, j\})$ una coalición de tamaño cualquiera, nuestra función de utilidad (3.14) es la suma sobre todos los jugadores del juego no pertenecientes a la coalición de una función de distancia. Llegando a la siguiente igualdad:

$$I_{i,j}^{Semi}(v^T) = \sum_{u \in V} I_{i,j}^{Semi}(v_u)$$

Si nos centramos en computar el siguiente término:

$$\sum_{C \in C^t(V \setminus \{i,j\})} \beta(t) \frac{S_{v_u}(C, i, j)}{\binom{|V|-2}{t}}, \quad t \in \{0, \dots, |V| - 2\}$$

iterando sobre todo posible tamaño de la coalición $t \in \{0, \dots, |V| - 2\}$ y el conjunto de vértices $u \in V$ del grafo, obtenemos el valor de $I_{i,j}^{Semi}(v^T)$.

Teorema 3.3.1 (Tarkowski et al. (2019)). *El índice de interacción $I_{i,j}^{Semi}(v^T)$ puede ser computado en $O(V_k^2|V|^2)$.*

Demostración. Sean $u, i, j \in V$ y $C \subseteq V \setminus \{u, i, j\}$, podemos suponer sin pérdida de la generalidad que $d(i, u) \leq d(j, u)$, dándose así dos posibles subcasos:

- $d(C, u) > d(i, u)$. En cuyo caso, $d(C \cup \{i, j\}, u) = d(i, u)$.
- $d(C, u) \leq d(i, u)$, donde tendríamos que $d(C \cup \{i, j\}) = d(C, u)$.

Teniéndose en cualquiera de ellos, $S_{v_u}(C, i, j) = -\Delta_j v_u(C)$, puesto que $\Delta_{\{i,j\}} v_u(C) = \Delta_i v_u(C)$.

En consecuencia, todo se centrará en encontrar la expresión de $\Delta_j v_u(C)$, donde j no es más que aquel nodo del par $\{i, j\}$ que cumple encontrarse a mayor distancia de $u \in V$. Consideremos de nuevo los dos escenarios anteriores.

- $d(C, u) > d(j, u)$ implica que $v_u(C \cup j) > v_u(C)$, luego $\Delta_j v_u(C) > 0$.
- $d(C, u) \leq d(j, u)$. En este caso $\Delta_j v_u(C) = 0$, ya que la distancia a u de la coalición $C \cup j$ será precisamente $d(C, u)$.

Llegando así a que

$$\sum_{C \in C^t(V \setminus \{i,j\})} \beta(t) \frac{S_{v_u}(C, i, j)}{\binom{|V|-2}{t}} = \sum_{C \in \left\{ C \in C^t(V \setminus \{i,j\}) \mid \begin{array}{l} | \\ d(C, u) > d(j, u) \end{array} \right\}} \beta(t) \frac{v_u(j) - v_u(C)}{\binom{|V|-2}{t}}$$

Separamos esta última expresión en dos partes:

$$\begin{aligned} MC^+(i, j, u, t) &= \sum_{C \in \left\{ C \in C^t(V \setminus \{i,j\}) \mid \begin{array}{l} | \\ d(C, u) > d(j, u) \end{array} \right\}} \beta(t) \frac{v_u(j)}{\binom{|V|-2}{t}} \\ MC^-(i, j, u, t) &= \sum_{C \in \left\{ C \in C^t(V \setminus \{i,j\}) \mid \begin{array}{l} | \\ d(C, u) > d(j, u) \end{array} \right\}} \beta(t) \frac{v_u(C)}{\binom{|V|-2}{t}} \end{aligned}$$

En el caso de la primera, conocemos $v_u(j) = \frac{1}{d(j,u)^2}$. Por lo cual, calculando $|\{C \in C^t(V \setminus \{i, j\}) \mid d(C, u) > d(j, u)\}|$, que fijémonos, es precisamente $\binom{Nodos_{>d}(u)}{t}$, tenemos:

$$MC^+(i, j, u, t) = \beta(t) \frac{1}{d(j, u)^2 \binom{|V|-2}{t}} \binom{Nodos_{>d}(u)}{t}$$

Mientras que en el caso de la segunda expresión, al no conocer la coalición C no sabemos cuál es el valor de $v_u(C)$. Sin embargo, ya que sabemos que $d(C, u) > d(j, u)$, siendo esta última una distancia que conocemos, obtenemos:

$$MC^-(i, j, u, t) = \sum_{\tilde{d} \in \left\{ \begin{array}{l} d \in \text{distancias}(u) \\ d > d(j, u) \end{array} \right\}} MC^-(i, j, u, t, \tilde{d})$$

donde

$$MC^-(i, j, u, t, \tilde{d}) = \sum_{C \in \left\{ \begin{array}{l} C \in C^t(V \setminus \{i, j\}) \\ d(C, u) = \tilde{d} \end{array} \right\}} \beta(t) \frac{v_u(C)}{\binom{|V|-2}{t}}$$

Dado que el número de coaliciones que verifica estar a distancia $\tilde{d}$ de u tendrá al menos un nodo que se encuentre a distancia $\tilde{d}$, llegamos a la siguiente formulación:

$$MC^-(i, j, u, t, \tilde{d}) = \left(\binom{Nodos_{\geq \tilde{d}}}{t} - \binom{Nodos_{> \tilde{d}}}{t} \right) \frac{1}{\tilde{d}^2 \binom{|V|-2}{t}}$$

Con esto, concluimos que el cálculo del índice de interacción asociado al semivalor v_f^{CL} se puede calcular como sigue:

$$I_{i,j}^{Semi}(v_f^{CL}) = \sum_{u \in V} \sum_{t=0}^{|V|-2} [MC^+(i, j, u, t) - MC^-(i, j, u, t)] \quad (3.15)$$

□

En el caso del cálculo de los índices de interacción definidos por Tarkowski et al. (2019), será necesario computar el cardinal de los conjuntos $Nodos_{\geq_d}(u)$, $Nodos_{>_d}(u)$. Para ello, se presenta el siguiente algoritmo, que está integrado en el cálculo del índice en el trabajo del autor. Sin embargo, aquí se expone por separado, en primer lugar, porque si queremos realizar múltiples experimentos sobre la misma red, lo más conveniente es almacenar la salida de estos cardinales, evitando así repetir el cálculo para cada conjunto de entrenamiento.

Este proceso tiene una complejidad temporal de $O(|V|V_k)$, ya que se itera sobre todos los vértices del grafo y, para cada uno, sobre los nodos dentro de su k -vecindad.

Algorithm 3: Cardinales de subconjuntos de nodos

Input: Grafo $\mathbf{G} = (\mathbf{V}, \mathbf{E})$: red de jugadores en forma de grafo,
 $\forall u \in V$: $\mathbf{distancias}_{\leq k}[\mathbf{u}]$ - diccionario de distancias al resto de nodos en orden descendente
Output: $\forall u \in V$: $\mathbf{subconjuntos}[\mathbf{u}]$ - diccionario tal que $\forall d \in \mathbf{distancias}_{\leq k}[\mathbf{u}]$ almacena $|\mathbf{Nodos}_{>d}(u)|$ y $|\mathbf{Nodos}_{\geq d}(u)|$

```
for  $u \in V$  do
     $\mathbf{subconjuntos}[u] \leftarrow$  diccionario vacío ;
     $h \leftarrow \max(\mathbf{distancias}[u])$  ;
     $\mathbf{Nodos}_{>h}(u) = |V| - |\mathbf{distancias}[u]|$  ;
     $\mathbf{Nodos}_{\geq h}(u) = |V| - |\mathbf{distancias}[u]|$  ;
    for  $d \in \mathbf{distancias}[u]$  do
        if  $d \neq h$  then
             $\mathbf{subconjuntos}[u][h] \leftarrow (\mathbf{Nodos}_{>h}(u), \mathbf{Nodos}_{\geq h}(u))$  ;
             $\mathbf{Nodos}_{>d}(u) \leftarrow \mathbf{Nodos}_{\geq h}(u)$  ;
             $h \leftarrow d$  ;
         $\mathbf{Nodos}_{\geq h}(u) \leftarrow \mathbf{Nodos}_{\geq h}(u) + 1$  ;
     $\mathbf{subconjuntos}[u][h] \leftarrow (\mathbf{Nodos}_{>h}(u), \mathbf{Nodos}_{\geq h}(u))$ 
```

A continuación se presenta el algoritmo para el cálculo de la expresión (3.15).

Algorithm 4: Índice de interacción relativo a un semivalor de Tarkowski

Input: $\mathbf{V}$: conjunto de vértices de la red, β : función de distribución de probabilidad discreta, $\mathbf{f}$:
función de distancia,
 $\forall u \in V$: $\mathbf{distancias}_{\leq k}[\mathbf{u}]$ - diccionario de distancias al resto de nodos en orden descendente,
 $\forall u \in V$: $\mathbf{subconjuntos}[\mathbf{u}]$ - diccionario tal que $\forall d \in \mathbf{distancias}_{\leq k}[\mathbf{u}]$ almacena $|\mathbf{Nodos}_{>d}(u)|$ y $|\mathbf{Nodos}_{\geq d}(u)|$
Output: $\mathbf{I}^{\text{Shapley}}$: matriz cuadrada estrictamente triangular inferior que almacena el índice de interacción entre todos los pares de nodos posibles

```
 $\mathbf{I}^{\text{Shapley}} \leftarrow$  matriz de ceros cuadrada ;
for  $u \in V$  do
    for  $t \in \{0, \dots, |V| - 2\}$  do
         $\text{lista\_dist} \leftarrow \mathbf{distancias}[u]$ ;
         $h \leftarrow \max(\mathbf{distancias}[u])$  ;
         $MC_t^- \leftarrow 0$  ;
        while  $|\text{lista\_dist}| > 1$  do
             $(i, d) \leftarrow$  primer elemento en  $\text{lista\_dist}$  ;
            Extraer  $(i, d)$  de  $\text{lista\_dist}$  ;
            if  $d \neq h$  then
                 $\mathbf{Nodos}_{<h}(u) \leftarrow |V| - \mathbf{Nodos}_{\geq h}(u)$  ;
                 $\mathbf{Nodos}_{\leq h}(u) \leftarrow |V| - \mathbf{Nodos}_{>h}(u)$  ;
                 $MC_t^- \leftarrow MC_t^- + \frac{\beta(t)f(h)}{\binom{|V|-2}{t}} ((\mathbf{Nodos}_{\geq h}^t(u)) - (\mathbf{Nodos}_{>h}^t(u)))$ ;
                 $h \leftarrow d$  ;
            for  $(j, dj) \in \text{lista\_dist}$  do
                 $\mathbf{Nodos}_{\leq d}(u) \leftarrow |V| - \mathbf{Nodos}_{>d}(u)$  ;
                 $MC_t^+ \leftarrow \frac{\beta(t)f(d)}{\binom{|V|-2}{t}} (\mathbf{Nodos}_{>d}^t(u))$  ;
                 $\mathbf{I}^{\text{Shapley}}[i, j] \leftarrow \mathbf{I}^{\text{Shapley}}[i, j] + MC_t^- - MC_t^+$ ;
```

Nota. Este algoritmo, en lugar de tomar cada par de nodos y después considerar cada nodo dentro

de su k -vecindad, invierte el procedimiento considerando todo nodo del grafo y después todo par de nodos dentro de su k -vecindad. Haciendo esto, evitamos los pares de vértices que se encuentran a mayor distancia que k del resto del grafo salvo para la inicialización de su distancia a infinito e índice de interacción a cero.

Siguiendo esta estrategia, se consigue un algoritmo que calcula los índices de interacción entre todo par de nodos del grafo en $O(|V|^2 V_k^2)$.

A través de un argumento similar al presentado en los teoremas 3.2.2 y 3.3.1, Tarkowski et al. (2019) propone el **índice de interacción de k -pasos amortiguado de Shapley**.

Algorithm 5: Índice de interacción de k -pasos amortiguado de Shapley

Input: V : conjunto de vértices de la red, f : función de distancia,
 $\forall u \in V$: **distancias** $_{\leq k}[u]$ - diccionario de distancias al resto de nodos en orden descendente,
 $\forall u \in V$: **subconjuntos** $[u]$ - diccionario tal que $\forall d \in \text{distancias}_{\leq k}[u]$ almacena $|Nodos_{>d}(u)|$ y $|Nodos_{\geq d}(u)|$
Output: I^{Shapley} : matriz cuadrada estrictamente triangular inferior que almacena el índice de interacción entre todos los pares de nodos posibles

$I^{\text{Shapley}} \leftarrow$ matriz de ceros cuadrada ;

for $u \in V$ **do**

$\text{lista_dist} \leftarrow \text{distancias}[u]$;

$h \leftarrow \text{máx}(\text{distancias}[u])$;

$MC^- \leftarrow 0$;

while $|\text{lista_dist}| > 1$ **do**

$(i, d) \leftarrow$ primer elemento en lista_dist ;

Extraer (i, d) de lista_dist ;

if $d \neq h$ **then**

$Nodos_{<h}(u) \leftarrow |V| - Nodos_{\geq h}(u)$;

$Nodos_{\leq h}(u) \leftarrow |V| - Nodos_{>h}(u)$;

$MC^- \leftarrow MC^- + \frac{f(h)}{Nodos_{<h}(u)-1} + \frac{f(h)}{Nodos_{\leq h}(u)-1}$;

$h \leftarrow d$;

for $(j, d_j) \in \text{lista_dist}$ **do**

$Nodos_{\leq d}(u) \leftarrow |V| - Nodos_{>d}(u)$;

$MC^+ \leftarrow \frac{f(d)}{Nodos_{\leq d}(u)-1}$;

$I^{\text{Shapley}}[i, j] \leftarrow I^{\text{Shapley}}[i, j] + MC^- - MC^+$;

Análogamente a lo que ocurre con el índice de **k -grado de Shapley**, la complejidad temporal del algoritmo 5 se reduce a $O(|V| V_k^2)$.

Capítulo 4

Predicción de enlaces

Las redes sociales son sistemas altamente dinámicos que evolucionan rápidamente con el tiempo mediante la incorporación de nuevas aristas. La aparición de estas aristas refleja nuevas interacciones en la estructura social subyacente, y los modelos que predicen su aparición tienen aplicaciones en diversos dominios. Algunos ejemplos incluyen sistemas de recomendación, utilizados en plataformas de *e-commerce*; la biología y la bioinformática, donde se analizan interacciones entre proteínas; e incluso campos relacionados con la seguridad, donde la predicción de vínculos puede emplearse para detectar grupos criminales.

Si bien en algunos casos la formación de un enlace entre dos agentes responde a factores exógenos difíciles de predecir, en la mayoría de ocasiones esta información puede inferirse a partir de las características estructurales de la red. En este trabajo, presentamos un enfoque basado exclusivamente en información topológica, modelando la red mediante un juego cooperativo, donde la función característica se define de manera endógena.

Una de las nociones más importantes en el contexto de redes es la *proximidad* entre vértices. Las técnicas de predicción de enlaces que se basan en esta idea asumen que, cuanto más próximos sean dos nodos en la red, mayor será la probabilidad de que exista un vínculo entre ellos. Desde una perspectiva sociológica, las personas tienden a establecer relaciones con individuos que comparten características como educación, religión, intereses o ubicación.

Estos modelos de predicción de vínculos asignan a cada par de nodos no conectados en la red una puntuación de semejanza, que representa la probabilidad de que en el futuro se establezca un enlace entre ellos. Como resultado, el modelo genera un ranking, en orden decreciente de puntuación, donde las aristas con mayor valor son aquellas más propensas a formarse y, por tanto, las principales candidatas a ser recomendadas.

4.1. Evaluación del rendimiento en predicción de enlaces

Una vez definidos los algoritmos para el cálculo de los distintos índices de interacción, procedemos a evaluar su rendimiento sobre redes reales. Seleccionaremos las siguientes cinco redes no dirigidas, ni ponderadas.

Karate Club. Se trata de la red de miembros de un club de Karate en una universidad de EE.UU. en 1977, las aristas representan relaciones.

Dolphins. Proviene de un estudio de observación de siete años sobre una comunidad de delfines en Nueva Zelanda. Una arista indica una asociación frecuente.

Nombre de la red	Número de vértices	Número de aristas
<i>Zachary Karate Club</i>	34	78
<i>Dolphins</i>	62	159
<i>Political books</i>	105	441
<i>Football</i>	115	613
<i>Jazz</i>	198	2742

Cuadro 4.1: Características de las redes empleadas.

Political books. Se trata de una red en la que los nodos representan libros sobre política de EE.UU. que fueron vendidos por *Amazon* durante las elecciones presidenciales de 2004. Una arista indica la existencia de un patrón frecuente de compra conjunta entre los usuarios.

Football. Es una red que simboliza partidos de fútbol americano universitario entre equipos de la División IA durante la temporada de otoño del año 2000.

Jazz. Representa músicos de Jazz. Una arista entre dos músicos denota que han tocado juntos en una misma banda.

Todas proceden de la página web de un proyecto nacido en la universidad de Klobenz-Landau, Alemania: The Konect Project. El objetivo será analizar si los índices de interacción propuestos aportan una mejora en la capacidad predictiva con respecto a otras medidas de similitud, introducidas a continuación.

4.1.1. Medidas de similitud para la comparación

En esta sección exploraremos algunas medidas de similitud conocidas con el propósito de evaluar y contrastar la precisión de las recomendaciones generadas por nuestros índices de interacción.

- **k-Common Neighbors (CN).** Se trata de una medida local que asigna a cada arista no existente entre un par de nodos, el número de nodos comunes en su *k-vecindad*.

$$CN^k(s, t) = |N_k(s) \cap N_k(t)|, \forall s, t \in V$$

- **k-Índice de Adamic-Adar (AAI).** Consiste en una adaptación del índice de *Lada Adamic* y *Eytan Adar* a las *k-vecindades*.

$$AAI^k(s, t) = \sum_{n \in (N_k(s) \cap N_k(t))} \frac{1}{\log |N_k(n)|}, \forall s, t \in V$$

4.1.2. Métricas de eficiencia

A continuación se presentarán ciertas adaptaciones de las métricas de rendimiento más utilizadas en problemas de clasificación binaria a nuestros algoritmos de predicción de vínculos, cuya salida es un *ranking*.

- **Área bajo la curva ROC.** La curva ROC normalmente suele trazar la tasa de verdaderos positivos frente a la tasa de falsos positivos. En nuestro caso, dada nuestra lista de clasificación de vínculos, el AUC refleja la probabilidad de que un vínculo aleatoriamente seleccionado del conjunto de vínculos eliminados tenga una posición más alta en la clasificación que un vínculo aleatoriamente seleccionado del conjunto de aristas no existentes en el grafo original. Siendo:

- m : número de vínculos *faltantes* o eliminadas.
- l : número de vínculos *no existentes* en el grafo original.
- $n = m \times l$: número de comparaciones totales.
- n' : número de comparaciones en las que el link *faltante* se encuentra en una posición más alta que el link *no existente*.
- n'' : número de comparaciones en las que los vínculos *faltante* y *existente* poseen la misma posición en el ranking.

Definimos el AUC como:

$$AUC = \frac{n' + 0,5n''}{n}$$

- **Precisión.** Sea m el número de vínculos faltantes. Si denotamos por $Top(m)$ a las m primeras aristas de la clasificación, y al subconjunto de estas aristas que se encuentra en el grafo original por $Correct(m)$, la precisión se formula como:

$$Precision = \frac{|Correct(m)|}{m}$$

Es decir, se trata del porcentaje de aristas faltantes que han sido clasificadas de manera correcta de acuerdo con los datos reales.

4.1.3. Diseño experimental

La evaluación del desempeño de los *índices de interacción* en el problema de la *link prediction* consistirá en ver si, creando un nuevo grafo a través de la eliminación de aristas reales, el modelo identifica estas relaciones como aristas con alta probabilidad de aparición. El siguiente esquema puede aplicarse a cualquier medida de similitud que asigne a dos vértices de la red un *score* de semejanza.

Notación. Sea $G = (V, E)$ el grafo que representa alguna de nuestras redes de trabajo:

- E^C representa conjunto de aristas no existentes en G , $E^C = (V \times V) \setminus E$.
- Denotamos por $S : V \times V \rightarrow \mathbb{R}$ a una función que me define un ranking sobre el conjunto de aristas. Todo índice de interacción es una aplicación de esta manera.
- $\mathcal{M}_S : G \rightarrow S(E^C)$ describe un modelo de predicción, el cual, a cada grafo le asigna un ranking ordenado de aristas no existentes.

Sea $G = (V, E)$ una red, $S : V \times V \rightarrow \mathbb{R}$ una función de similitud que proporciona una clasificación de aristas, y $\mathcal{M}_S : G \rightarrow S(E^C)$ un modelo de predicción definido por una función de similitud, presentamos a continuación el algoritmo que emplearemos para la evaluación.

Explicación:

1. Reiteraremos 100 veces el siguiente proceso, almacenando en cada una de las iteraciones los resultados obtenidos para las métricas de evaluación que serán el *AUC* y la *Precisión* (explicados en la siguiente sección).
2. Retiramos aleatoriamente el 30 % de las aristas para obtener el grafo de entrenamiento $G^T = (V, E^T)$.
3. Aplicamos el modelo de predicción y obtenemos un *ranking* de aristas recomendadas, incluyendo las que fueron eliminadas.

4. Evaluamos las métricas de rendimiento y almacenamos sus valores en una lista.
5. Finalmente devolvemos la media de las repeticiones de las métricas.

Algoritmo en pseudocódigo:

Algorithm 6: Rendimiento en predicción de vínculos

Input: Grafo $G = (V, E)$, $\mathcal{M}_S : G \rightarrow S(E^C)$ modelo de predicción

Output: $(AUC_{aver}, Precision_{aver})$

$AUC_{aver}, Precision_{aver} \leftarrow 0, 0$

for $i = 1$ **to** 100 **do**

$E_{rem} \leftarrow Sample(E, 0, 3|E|)$

$E^T \leftarrow E \setminus E_{rem}$

$G^T \leftarrow (V, E^T)$

$Ranking \leftarrow \mathcal{M}(G^T)$

$AUC_{aver} \leftarrow AUC(Ranking)$

$Precision_{aver} \leftarrow Precision(Ranking)$

$AUC_{aver} \leftarrow \frac{AUC_{aver}}{100}$

$Precision_{aver} \leftarrow \frac{Precision_{aver}}{100}$

return: $(AUC_{aver}, Precision_{aver})$

4.2. Experimento por etapas

En esta sección describimos un experimento diseñado para analizar el impacto de la inserción progresiva de aristas en la precisión de las clasificaciones generadas por las medidas locales de similitud. Inicialmente, se obtiene el ranking de predicción de aristas utilizado en los apartados anteriores y, posteriormente, las aristas mejor clasificadas se insertan en la red una por una, reevaluando la similitud entre nodos tras cada incorporación (A).

	k	SVII		SVMII		CNN	
		Original	Modificada	Original	Modificada	Original	Modificada
Zachary	1	232.61	265.22	234.78	232.61	163.04	173.91
	2	204.35	191.3	117.39	110.87	69.56	86.95
	3	232.61	215.22	121.74	113.04	147.82	134.78
Dolphins	1	195.83	150	173.96	148.96	204.17	135.41
	2	203.12	160.42	182.29	141.67	127.08	97.92
	3	197.92	160.42	127.08	98.96	100	76.04
Polbooks	1	292.05	282.2	292.05	282.2	292.05	282.2
	2	304.16	279.92	226.52	209.47	217.045	188.26
	3	295.45	268.56	295.45	268.56	135.99	68.94
Football	1	400.8	403.26	405.44	405.16	402.17	386.68
	2	398.1	391.3	214.13	36.41	173.64	42.39
	3	387.23	386.41	35.05	29.07	30.43	26.08

Cuadro 4.2: Resultados del experimento por etapas.

Los resultados se encuentran en la Tabla 4.2, la cual muestra los valores de precisión original y precisión modificada. Como podemos observar, en la mayoría de los casos seguir un procedimiento por etapas empeora la precisión del modelo de predicción.

Capítulo 5

Detección de comunidades

Las redes reales suelen exhibir un carácter altamente heterogéneo, reflejando elevados niveles de organización y orden. Se dice que una red tiene **estructura de comunidad** cuando en ella se identifican grupos de vértices altamente conectados, con una densidad de aristas elevada en su interior y pocas conexiones con el resto de la red.

Las comunidades pueden, hasta cierto punto, considerarse entidades separadas con su propia autonomía. Su estudio permite comprender mejor la estructura y el funcionamiento de las redes, facilitando algunas de las siguientes tareas:

- **Identificación de grupos de nodos con características similares.** En el ámbito del marketing esta tarea es fundamental para sistemas de recomendación y segmentación de audiencias.
- **Clasificación de nodos según su posición estructural.** Por ejemplo, aquellos vértices más centrales en una red pueden desempeñar un papel crucial en la estabilidad del grupo, mientras que los nodos periféricos actúan como intermediarios entre distintas comunidades, facilitando la difusión de información.
- **Reconocimiento de estructuras jerárquicas y organizativas.** Todas las redes que nos rodean están constituidas a través de la organización jerárquica de subcomunidades, y parecer no ser casualidad. El economista Simon (1962) defendía la idea de que este tipo de estructuras desempeñan un rol fundamental en la rápida evolución de sistemas complejos.

El problema de la identificación de comunidades en grafos busca extraer información topológica de una red con el fin de encontrar una partición de nodos, que en lo que a nosotros respecta, será disjunta.

5.1. Clustering jerárquico e índices de interacción

Sea $G = (V, E)$ una red de agentes. Un algoritmo de clustering **aglomerativo** es un procedimiento de identificación de comunidades que comienza con $|V|$ comunidades distintas, cada una compuesta por un único nodo, y en cada iteración fusiona aquellas comunidades más *cercanas* o *similares*. Este proceso de clusterización crea una jerarquía de comunidades, de manera que, sean $C_1, C_2 \subseteq V$ dos comunidades formadas en el transcurso de este algoritmo se tiene que $C_1 \cap C_2 = \emptyset$, $C_1 \subseteq C_2$ o bien $C_2 \subseteq C_1$.

Para comprender mejor cómo se toma la decisión de unificar dos comunidades, es necesario definir qué se entiende por *comunidades cercanas o similares*. En general, se considera que dos

comunidades son cercanas si, al utilizar una función de distancia entre conjuntos de nodos, su separación es pequeña. Dada una función $d : (V \times V) \rightarrow \mathbb{R}$, que mide la proximidad entre pares de nodos, es posible extender este concepto al caso de subconjuntos de vértices o comunidades adoptando diferentes enfoques.

- **Single linkage** o *nearest neighbor linkage*. Define la distancia entre dos comunidades como la menor distancia entre dos nodos de cada cluster. Es decir, sean $C_1, C_2 \subseteq V$; $C_1 \cap C_2 = \emptyset$ y $C_1, C_2 \neq \emptyset$:

$$d(C_1, C_2) = \min_{(i,j) \in C_1 \times C_2} d(i, j)$$

- **Complete linkage** o *farthest neighbor linkage*. La distancia entre dos comunidades es tomada como la mayor distancia entre dos nodos de cada cluster. Luego,

$$d(C_1, C_2) = \max_{(i,j) \in C_1 \times C_2} d(i, j)$$

- **Average linkage**. Como su nombre indica la distancia entre dos comunidades es determinada por la media de las distancias entre cada posible par de nodos de cada cluster.

$$d(C_1, C_2) = \frac{1}{|C_1||C_2|} \sum_{i \in C_1} \sum_{j \in C_2} d(i, j)$$

Cada una de estas técnicas genera divisiones en comunidades distintas. El primer enfoque produce clústeres alargados o con estructura dispersa, siendo más sensibles a *outliers*. Esto ocurre porque, aunque dos comunidades pueden contener nodos muy cercanos, también pueden incluir pares de nodos considerablemente alejados. En contraste, la segunda definición favorece la formación de comunidades compactas, asegurando una menor variabilidad en la distancia entre los nodos de una misma comunidad.

Observación 5.1.1. La función de distancia $d : V \times V \rightarrow \mathbb{R}$ no es más que una particularización del concepto de **disimilitud** entre dos nodos, ya que, cuanto más cercanos están, menor es su diferencia estructural.

Es natural asumir que las comunidades son grupos de vértices similares entre sí o con roles equivalentes dentro de la red. En este sentido, la similitud entre vértices puede calcularse en función de determinadas características y utilizarse para identificar aquellas comunidades con mayor probabilidad de aparición. Sin embargo, dado que la similitud es el **concepto opuesto a la disimilitud**, las nociones de linkage obtenidas anteriormente deben reformularse adecuadamente al trabajar con funciones que reflejan la semejanza.

Sea V el conjunto de jugadores de un juego cooperativo, y v su función característica, vamos a utilizar los índices de interacción como medida de similitud entre comunidades. Llevaremos a cabo un experimento de clustering jerárquico (*bottom-up*) tomando el *complete linkage* como técnica de agrupamiento. Por lo tanto, en cada paso fusionaremos aquellas comunidades C_1, C_2 con mayor índice de interacción, siendo este definido como:

$$II_{C_1, C_2}(v) = \max_{(i,j) \in C_1 \times C_2} II_{i,j}(v)$$

La mayoría de los algoritmos que siguen este enfoque requieren de un criterio de parada, pero en nuestro caso exploraremos todas las posibles divisiones hasta alcanzar un único clúster que agrupe todos los nodos del grafo. Para identificar la partición que mejor refleja la estructura de la red, emplearemos el concepto de *modularidad* definido en la siguiente sección.

5.2. Evaluación del rendimiento en detección de comunidades

En este trabajo, además de replicar el procedimiento de clustering desarrollado por Szczepanski et al. (2015), vamos a aplicar el **índice de interacción de k-pasos amortiguado de Shapley** a las redes de estudio. Nuestro objetivo es evaluar si, además de mejorar los resultados en la predicción de vínculos, este índice permite obtener comunidades más representativas de la estructura real de la red. Para una comparación más completa, contrastaremos estos dos índices de interacción con otros algoritmos de clustering ampliamente conocidos.

Una de las redes utilizadas en este estudio es la red *Zachary Karate Club*, un referente en la evaluación de algoritmos de clustering. Este grafo, introducido por el antropólogo Wayne Zachary, proviene de un estudio sobre la dinámica social de un club de karate en EE.UU., observado durante tres años. A raíz de un conflicto entre dos miembros clave, el club se dividió en dos grupos, lo que convierte a esta red en un caso excepcional donde se conoce la partición real de la comunidad.

5.2.1. Algoritmos de comparación

Entre los algoritmos utilizados para la comparación del rendimiento, se encuentra el propio algoritmo de *clustering jerárquico aglomerativo*, que emplea como criterio de fusión la medida de similitud *k-Common Neighbors* presentada en la Sección 4.1.1. Además, con el objetivo de realizar una comparación más exhaustiva, incorporamos dos algoritmos adicionales basados en la optimización de la modularidad.

- **Greedy modularity algorithm.** Se trata de una heurística voraz propuesta en Clauset et al. (2004) basada en *clustering aglomerativo*, cuyo objetivo es encontrar una partición del grafo que maximice la modularidad. Para ello, sigue este esquema:
 1. Se comienza con una partición donde cada nodo pertenece a su propia comunidad.
 2. En cada iteración, se evalúan todas las posibles fusiones entre pares de comunidades y se selecciona aquella que genere el mayor incremento en la modularidad.
 3. Se repite este proceso hasta que ninguna fusión entre comunidades produzca una mejora en la modularidad.
 4. La última partición obtenida es la salida del algoritmo.
- **Algoritmo de Louvain.** Este algoritmo es también un método de optimización voraz desarrollado por Blondel et al. (2008) en la Universidad de Louvain, de donde toma su nombre. Esta heurística se compone de dos fases:

Fase de optimización local:

1. Se asigna cada nodo a una comunidad propia.
2. Para cada nodo $u \in V$, se evalúa el cambio en la modularidad al moverlo a cada comunidad vecina, entendiendo por vecina, una comunidad a la que el nodo esté conectado.
3. Si existe algún movimiento que aumente la modularidad, el nodo se traslada a la comunidad que maximiza este incremento.
4. Repetimos este proceso hasta que ningún nodo pueda cambiar de comunidad mejorando la modularidad.

Fase de agregación:

1. Se crea un grafo donde cada comunidad detectada en la fase anterior se trata como un solo nodo.
2. Aplicamos a este nuevo grafo el procedimiento de la fase de optimización local.

Este proceso finaliza cuando no se pueden realizar más fusiones que aumenten la modularidad, obteniendo la partición óptima encontrada por la heurística.

Estas dos heurísticas ya se encuentran implementadas en la librería *NetworkX* de Python, por lo que haremos uso de sus funciones predefinidas en lugar de programarlas desde cero.

5.2.2. Métricas de eficiencia

Una **comunidad** es un grupo de nodos cohesionado en comparación con el resto de la red. Es por ello que la calidad de una división en comunidades viene determinada por la diferencia entre el número de aristas intracluster, y el número de aristas intercluster. Introduciremos a continuación las técnicas que emplearemos para la determinación de la calidad de nuestras divisiones, que serán adaptadas al caso de grafos sin dirección y pesos. Sea $G = (V, E)$ nuestra red de trabajo, y $\mathcal{C} := \{C_1, \dots, C_m\}$ una partición en comunidades tal que $\bigcup_{i=1}^m C_i = V$ y $C_i \cap C_j = \emptyset \forall i, j \in \{1, \dots, m\}, i \neq j$

- **Modularidad** (Q). Esta medida fue definida por Newman (2006) representa qué tan bien están conectados los nodos dentro de una división en comunidades, en comparación con lo que cabría esperar si las conexiones (aristas) de la red fueran asignadas de manera aleatoria. Sean $s, t \in V$ dos nodos con grados $\deg(s)$, $\deg(t)$ en la red G , respectivamente. Denotamos por P_{st} la probabilidad de que una arista caiga entre los vértices s y t y A_{st} el número real de aristas entre estos vértices s, t . La modularidad se define como:

$$Q(\mathcal{C}) = \frac{1}{2m} \sum_{st} [A_{st} - P_{st}] \delta(C_s, C_t)$$

dónde δ es la **función delta de Kronecker**, definida como $\delta(i, j) = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$.

Sin embargo, como desconocemos la distribución aleatoria de las aristas, el término P_{st} no está determinado. Para abordar este problema, consideramos ciertas suposiciones que un grafo aleatorio debería cumplir, establecidas en Clauset et al. (2004). Bajo estas condiciones, Newman logró formular una función de beneficio explícita para cada partición del grafo en subcomunidades. El grafo aleatorio debe cumplir lo siguiente:

- **Mantener el mismo número de vértices** $|V| = n$ que el grafo original.
- **Simetría en la probabilidad de conexión:** $P_{st} = P_{ts}$, al estar tratando exclusivamente con grafos no dirigidos.
- $Q(\mathcal{C}) = 0$ en el caso en que $\mathcal{C} = \{V\}$. Esto nos llevaría a que $\sum_{st} [A_{st} - P_{st}] = 0$, por lo cual, $\sum_{st} P_{st} = \sum_{st} A_{st} = 2m$. Teniendo de esta manera que el número esperado de aristas en toda la red es igual que el número de aristas en el grafo original.
- **El grado esperado de cada vértice dentro del grafo aleatorio debe coincidir con su grado en la red original:** $\sum_t P_{st} = \deg(s) \forall s \in V$.

- **Las probabilidades de conexión con un vértice $s \in V$ depende única y exclusivamente de $\deg(s)$:** $P_{st} = f(\deg(s))f(\deg(t))$. Esto implica que las probabilidades de que una arista una dos extremos son independientes.

Todas estas condiciones llevan a la ecuación $\sum_{t=1}^n P_{st} = f(\deg(s)) \sum_{t=1}^n f(\deg(t)) = \deg(s)$. De aquí, obtenemos que $f(\deg(s)) = k\deg(s)$, para alguna constante k . Sustituyendo en la ecuación total de probabilidad:

$$2m = \sum_{st} P_{st} = k^2 \sum_{st} \deg(s)\deg(t) = (2mk)^2$$

Luego $k = \frac{1}{\sqrt{2m}}$, y la probabilidad final de conexión es $P_{st} = \frac{\deg(s)\deg(t)}{2m}$. Llegamos así a nuestra expresión de modularidad:

$$Q(\mathcal{C}) = \frac{1}{2m} \sum_{st} \left[A_{st} - \frac{\deg(s)\deg(t)}{2m} \right] \delta(C_s, C_t)$$

- **Coverage (γ).** Denotamos por $\gamma(\mathcal{C})$ el coverage de una partición $\mathcal{C}$, el cual mide la proporción de aristas intra-cluster con respecto al total de aristas del grafo. Es decir,

$$\gamma(\mathcal{C}) := \frac{\sum_{i=1}^m |E(C_i)|}{|E|}, \quad E(C_i) := \{(i, j) : i, j \in C_i\}$$

Esta métrica, sin embargo, puede ser engañosa en algunos casos. Por ejemplo, si la partición es trivial, $\mathcal{C} = \{V\}$, el *coverage* siempre tomará valor 1 sin importar la estructura real del grafo. Además, maximizar el *coverage* es equivalente a minimizar el número de aristas inter-cluster, lo que se relaciona con la búsqueda de un *minimum cut*. Sin embargo, un *minimum cut* generalmente no representa una partición en comunidades realista, sino una separación basada en minimizar el costo del corte. Por lo tanto, el *coverage* suele utilizarse con otras métricas para obtener una evaluación más completa de la calidad de la partición.

Teniendo en cuenta estas definiciones, los valores de estas métricas de rendimiento oscilan dentro de los siguientes intervalos: $\gamma(\mathcal{C}) \in [0, 1]$, $Q(\mathcal{C}) \in [-\frac{1}{2}, 1]$.

5.2.3. Diseño experimental

En esta sección explicaremos detalladamente cómo se realiza el proceso de detección de comunidades seguido en el código del Apéndice A para cualquier medida de similitud entre dos comunidades de nodos.

Explicación:

1. Crearemos una lista en la que guardaremos el valor de la *modularidad* y el *coverage* de cada una de las particiones de la red en comunidades.
2. Inicializamos la primera partición $\mathcal{C}_1 = \{\{v\} \mid \forall v \in V\}$, en la cual cada nodo representa una comunidad individual. Añadiendo el valor de su modularidad y coverage a la lista de métricas.
3. Repetimos el siguiente proceso para toda partición hasta alcanzar la partición trivial $\mathcal{C}_{|V|-1} = \{V\}$:

- a) Encontramos en $\mathcal{C}_i$ las dos coaliciones más similares a través de la medida de similitud $\mathcal{S}$. Fusionamos las coaliciones y creamos así una nueva configuración de comunidades $\mathcal{C}_{i+1}$ tal que $|\mathcal{C}_{i+1}| = |\mathcal{C}_i| - 1$.
 - b) Almacenamos $Q(\mathcal{C}_{i+1}), \gamma(\mathcal{C}_{i+1})$.
4. Tomamos aquella configuración $\mathcal{C}_i$, $i \in \{1, \dots, |V| - 1\}$ con mayor modularidad de todas.

Algoritmo en pseudocódigo:

Algorithm 7: Rendimiento en detección de comunidades

Input: Grafo $G = (V, E)$, $\mathcal{S} : 2^V \times 2^V \rightarrow \mathbb{R}$ función de similitud de comunidades

Output: $(\mathcal{C}_k, Q(\mathcal{C}_k))$, $k := \arg \max_{l \in \{1, \dots, m\}} Q(\mathcal{C}_l)$

$Qlist, \gamma list \leftarrow [], []$;

$\mathcal{C}_1 \leftarrow \{v \mid v \in V\}$;

for $i = 2, \dots, |V| - 1$ **do**

$C_i, C_j \leftarrow \arg \max_{(C_1, C_2) \in \mathcal{C}_i \times \mathcal{C}_i} S(C_1, C_2)$;
 $C_{i \cup j} \leftarrow C_i \cup C_j$;
 $\mathcal{C}_i \leftarrow (\mathcal{C}_i \setminus \{C_1, C_2\}) \cup C_{i \cup j}$;
 $Qlist \leftarrow Qlist \cup Q(\mathcal{C}_i)$;
 $\gamma list \leftarrow \gamma list \cup \gamma(\mathcal{C}_i)$;

$k \leftarrow \arg \max_{l \in \{1, \dots, m\}} Q(\mathcal{C}_l)$;

return $(\mathcal{C}_k, Q(\mathcal{C}_k))$

Capítulo 6

Resultados experimentales y conclusiones

En este capítulo, se adoptará la siguiente convención para denotar las distintas medidas de similitud basadas en las k -vecindades.

- *SVII*: Índice de interacción de k -grado de Shapley.
- *SVMII*: Índice de interacción de k -pasos amortiguado de Shapley.
- *CNN*: k -vecinos más cercanos.
- *AAI*: Índice de Adamic-Adar.

Además, para cada uno de los problemas presentados, proporcionaremos los resultados correspondientes a los radios $k \in \{1, 2, 3\}$.

Cabe mencionar que, en el caso en que $k = 1$, las funciones características de k -grado de Shapley y de k -pasos amortiguado de Shapley son iguales, luego los índices de interacción que de ellas derivan también deberían serlo. Sin embargo, el método seguido para el cálculo de los cardinales de los conjuntos $Nodos_{\sim d}(u)$, donde $\sim$ es una relación de orden entre $\leq, <$, contabiliza también al propio nodo u . Mientras que, en el caso del índice de k -grado de Shapley, $deg_k(u)$ es el cardinal de la k -esfera de influencia. Por consiguiente, estos índices de interacción difieren en su valor para todos los posibles valores de k .

6.1. Predicción de vínculos

Retomando las condiciones experimentales definidas en la Sección 4.1, presentamos a continuación los resultados obtenidos para los índices indicados al inicio del capítulo. No obstante, los índices de interacción derivados de los semivalores de *Shapley* y *Banzhaf* también fueron empleados en la tarea de predicción de vínculos (Apéndice A). Estos muestran un rendimiento muy similar a los índices SVII y SVMII; sin embargo, debido a su mayor costo computacional, no se incluyen en esta sección.

En la Tabla 6.1 se muestra el rendimiento de nuestros índices de interacción como medidas de similitud para distintos valores de $k \in \mathbb{N}$ siguiendo el enfoque de Tarkowski et al. (2019). En particular, adoptamos el mismo número de aristas a eliminar para la formación del grafo de entrenamiento. A pesar de no haber repetido el experimento el mismo número de veces para calcular

la media de las métricas, los resultados obtenidos son muy similares a los reportados en dicho trabajo, donde el procedimiento se repite 1000 veces.

Sea una red $G = (V, E)$ y un radio $k \in \{1, 2, 3\}$ de la vecindad, resaltamos el mayor valor del área bajo la curva ROC y precisión. Además, los mejores valores de estas métricas obtenidos para cada red se presentan en un tono más oscuro.

	k	SVII		SVMII		CNN		AAI	
		AUC	Precisión	AUC	Precisión	AUC	Precisión	AUC	Precisión
Zachary	1	657.06	241.74	654.64	223.04	637.36	149.13	655.08	204.35
	2	638.53	94.78	673.97	193.48	596.25	73.91	604.74	65.65
	3	627.79	145.65	687.21	202.61	613.27	125.65	615.87	127.83
Dolphins	1	710.9	180.83	712.05	181.46	713.81	211.04	715.31	195.21
	2	770.43	188.33	765.41	198.33	743.52	117.5	755.86	134.37
	3	750.89	112.08	771.21	203.54	715.06	92.29	723.9	98.75
Polbooks	1	839.81	293.94	841.62	296.36	833.27	268.71	835.44	287.65
	2	858.05	222.73	880.25	295.98	835.57	214.24	838.43	221.82
	3	811.6	144.92	875.09	295.91	793.11	131.97	799.22	129.85
Football	1	813.73	405.54	813.09	410.33	814.27	405.16	813.77	412.07
	2	805.84	200.33	827.24	404.95	782.87	175.92	786.14	179.02
	3	550.59	36.09	816.04	402.07	532.87	34.29	536.71	31.25
Jazz	1	957.33	636.67	958.57	635.71	943.69	572.5	952.1	602.92
	2	843.46	282.41	945.86	607.74	803.65	245.71	808.97	251.97
	3	743.45	199.61	949.12	620.74	728.44	188.76	728.19	184.29

Cuadro 6.1: Precisión y AUC de las medidas de similitud.

A partir de estos resultados, se observa que el índice de interacción de **k-pasos amortiguado** es el más competitivo de los cuatro. Asimismo, este índice presenta la ventaja de no generar variaciones significativas en los resultados al modificar el radio k . Por ejemplo, si analizamos los valores de precisión obtenidos para la red *Jazz*, se puede apreciar un descenso de la tasa de aristas acertadas que oscila entre los 400/500 puntos para el resto de medidas, mientras que en el caso de SVMII, esta métrica fluctúa en menos de 30 puntos.

En cuanto al área bajo la curva ROC, aunque el aumento del radio también produzca un descenso de la métrica para las medidas SVII, CNN y AAI, este resulta menor.

La consistencia que este índice muestra en la tasa de aristas acertadas para la recomendación nos sugiere que el *principio de decrecimiento de influencia en la red*, aplicado por este método, le permite filtrar el ruido provocado por agentes que son más lejanos. Esto respalda la idea de que los patrones de vínculos emergentes están efectivamente influenciados por la estructura local de la red, reforzando el papel del entorno más cercano en la predicción de vínculos.

Finalmente, conviene señalar que los descensos en el rendimiento de las medidas de similitud con respecto a ambas métricas dependen de la topología de la red. Las redes analizadas no presentan el mismo grado de conexión, como se muestra en la Tabla 6.2.

	Zachary	Dolphins	Polbooks	Football	Jazz
Aristas/Jugadores	2.29	2.56	4.2	5.33	13.84

Cuadro 6.2: Ejemplo de tabla con dos filas y cinco columnas

Esta tabla pone de manifiesto que el grado de conectividad de la red influye en la cantidad de ruido presente en la predicción. En concreto, redes como *Zachary* y *Dolphins*, con menor cantidad de aristas por jugador, presentan una menor disminución en la eficiencia de sus predicciones. Por el contrario, en grafos como *Jazz* y *Football*, donde la densidad de conexiones es mayor, el desempeño de las medidas de similitud más susceptibles al ruido se ve significativamente afectado.

6.2. Detección de comunidades

En esta sección, a diferencia de la anterior, abordaremos el problema de detección de comunidades en todas las redes presentadas en 4.1, excepto en la red *Jazz*. Tal y como se indicó en 5.2, además de realizar una comparación con la medida de similitud k -vecinos más cercanos, se contrastarán los resultados con aquellos obtenidos por las heurísticas *Greedy Modularity* y *Louvain*.

De acuerdo con los resultados mostrados en la Tabla 6.3, la medida de similitud con mayores valores de modularidad en cualquiera de las redes se trata del índice de interacción de k -pasos amortiguado de Shapley. Estos valores de modularidad superan los logrados por el algoritmo voraz denominado *Greedy* y se acercan a los obtenidos por *Louvain* en la mayoría de los grafos salvo *Dolphins*, lo cual hace de esta medida una técnica más competente de detección de comunidades (ver 6.4).

	k	SVII			SVMII			CNN		
		MOD COV número			MOD COV número			MOD COV número		
Zachary	1	391.44	628.21	5	411.16	730.77	4	121.63	474.36	5
	2	313.28	846.15	2	356.92	858.97	2	313.28	846.15	2
	3	77.75	346.15	19	411.16	730.77	4	77.75	346.15	19
Dolphins	1	381.72	528.3	16	347.53	465.41	16	262.93	408.81	22
	2	447.04	679.24	5	433.9	698.11	5	464.38	811.32	5
	3	368.97	924.53	2	467.19	779.87	4	304.24	930.82	2
Polbooks	1	388.36	632.65	12	383.42	639.46	11	312.3	537.41	14
	2	445.14	945.58	2	520.11	907.03	4	445.14	945.58	2
	3	442.23	918.37	4	520.11	907.03	4	437.94	938.77	2
Football	1	600.5	690.05	12	600.52	690.05	12	601.01	691.68	12
	2	564.55	686.78	9	604.43	706.36	10	565.85	768.35	5
	3	104.99	636.22	8	602.91	704.73	10	84.8	528.55	22

Cuadro 6.3: Modularidad, coverage y número de comunidades de las medidas de similitud.

Red	Greedy		Louvain	
	MOD	número	MOD	número
Zachary	380.67	3	415.11	4
Dolphins	495.49	4	523.38	5
Polbooks	501.97	4	526.79	5
Football	568.24	6	604.57	10

Cuadro 6.4: Resultados Greedy Modularity y Louvain.

Por otro lado, cabe recordar que estas métricas de rendimiento evalúan diferentes aspectos de la calidad de las particiones. En general, la modularidad suele considerarse la métrica más representativa de una buena división en comunidades, ya que sigue el principio de densidad proporcionado por la definición de comunidad 5.2.2.

Sin embargo, en el caso de la red *Zachary*, las particiones con mayor coverage se asemejan más a la división original de la red 6.1. Las particiones obtenidas mediante los índices de interacción, aunque no coinciden exactamente con la estructura original, generan divisiones muy similares. En ambos casos, ciertos nodos son asignados al grupo contrario; sin embargo, todos ellos- 9 y 20 en la partición generada con SVMII y 3, 4, 9, 20 en la partición generada con SVII- corresponden a nodos con múltiples conexiones en ambos grupos, lo que indica que ocupan posiciones altamente centrales en la red.

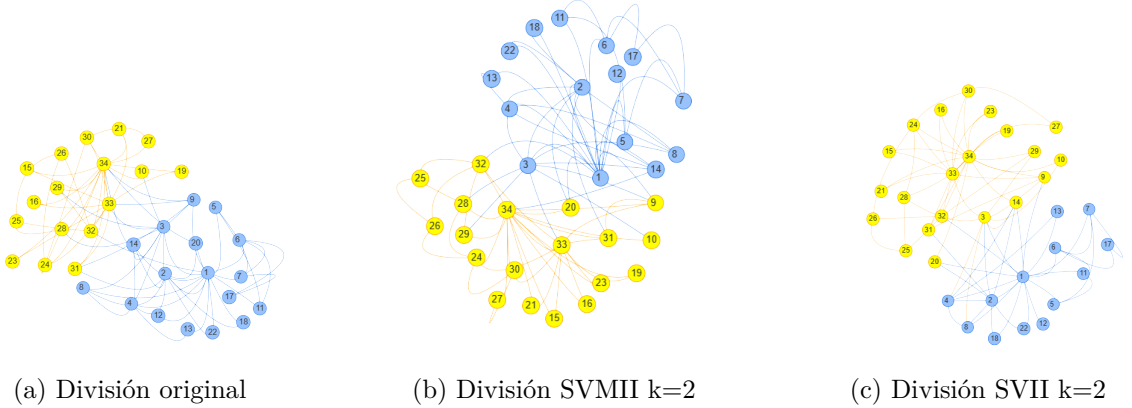


Figura 6.1: Particiones grafo Zachary en dos comunidades

De manera similar a lo observado en el problema de predicción de vínculos, los índices SVII y CNN suelen obtener peores resultados a medida que aumenta el radio de la esfera de influencia, como se observa en las redes *Zachary* y *Football*. En contraste, el algoritmo de clustering aglomerativo basado en SVMII muestra un comportamiento opuesto, logrando, en la mayoría de los casos, mejores valores de modularidad y coverage para radios más elevados.

A continuación se expone un ejemplo correspondiente a las particiones generadas para $k=3$ con ambos índices y la obtenida a través de la heurística de Louvain. Mostramos también en la Figura 6.3 el proceso de clustering aglomerativo que se sigue con estos índices a través de su dendrograma. Todas estas figuras han sido diseñadas en el Apéndice A.

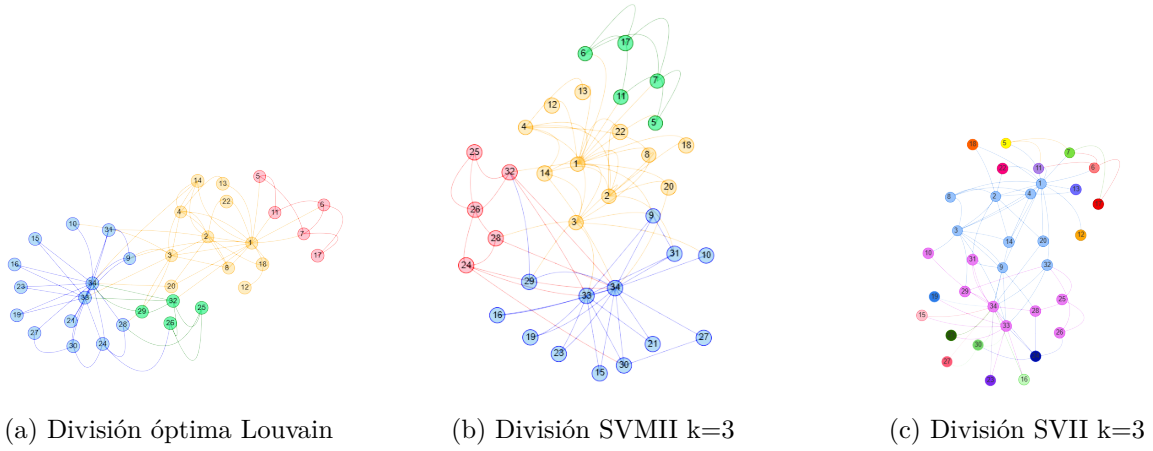


Figura 6.2: Particiones grafo Zachary comparación Louvain

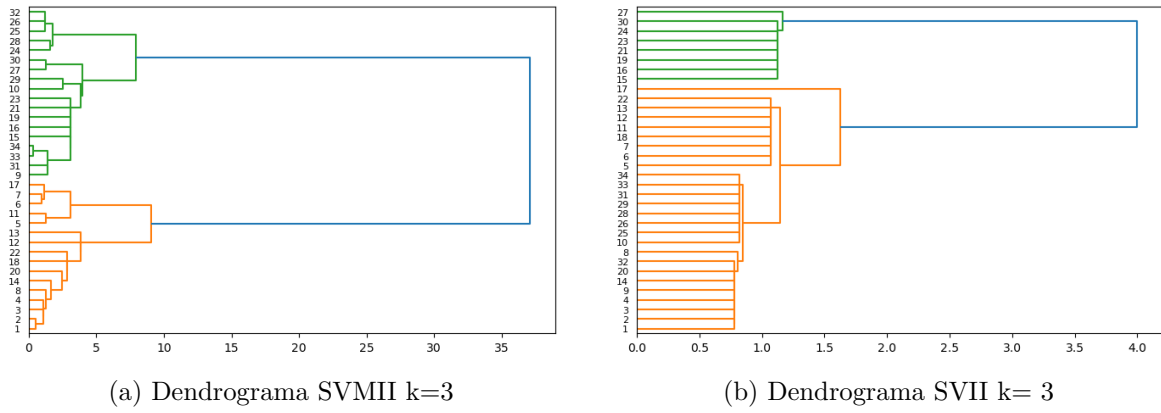


Figura 6.3: Proceso de clustering aglomerativo para SVMII y SVII con $k=3$

Bibliografia

- Banzhaf III, J. F. (1965), ‘Weighted voting doesn’t work: A mathematical analysis’, *Rutgers Law Review* **19**, 317–343.
- Blondel, V., Guillaume, J.-L., Lambiotte, R. & Lefebvre, E. (2008), ‘Fast unfolding of communities in large networks’, *Journal of Statistical Mechanics Theory and Experiment* **2008**.
- Clauset, A., Newman, M. E. J. & Moore, C. (2004), ‘Finding community structure in very large networks’, *Phys. Rev. E* **70**, 066111.
- Dubey, P., Neyman, A. & Weber, R. J. (1981), ‘Value theory without efficiency’, *Mathematics of Operations Research* **6**(1), 122–128.
URL: <http://www.jstor.org/stable/3689271>
- Feltkamp, V. (1995), ‘Alternative axiomatic characterizations of the shapley and banzhaf values’, *Int. J. Game Theory* **24**(2), 179–186.
- Fujimoto, K., Kojadinovic, I. & Marichal, J.-L. (2006), ‘Axiomatic characterizations of probabilistic and cardinal-probabilistic interaction indices’, *Games and Economic Behavior* **55**(1), 72–99.
URL: <https://www.sciencedirect.com/science/article/pii/S0899825605000278>
- Gillies, D. (1953), Some Theorems on n-Person Games, PhD thesis, Princeton University.
- Girvan, M. & Newman, M. E. J. (2002), ‘Community structure in social and biological networks’, *Proceedings of the National Academy of Sciences* **99**(12), 7821–7826.
- Grabisch, M. (1997), ‘k-order additive discrete fuzzy measures and their representation’, *Fuzzy Sets and Systems* **92**(2), 167–189. Fuzzy Measures and Integrals.
URL: <https://www.sciencedirect.com/science/article/pii/S0165011497001681>
- Grabisch, M. & Roubens, M. (1999), ‘“an axiomatic approach to the concept of interaction among players in cooperative games”’, *International Journal of Game Theory* **28**, 547–565.
- Lange, F. (2007), Exploration de la valeur de shapley et des indices d’interaction pour les jeux définis sur des ensembles ordonnés.
- Liben-Nowell, D. & Kleinberg, J. (2003), The link prediction problem for social networks, Association for Computing Machinery, New York, NY, USA.
- Newman, M. E. J. (2006), ‘Finding community structure in networks using the eigenvectors of matrices’, *Phys. Rev. E* **74**, 036104.
- Owen, G. (1972), ‘Multilinear extensions of games’, *Management Science* **18**, 64–79.

- Roubens, M. (1996), Interaction between criteria and definition of weights in mcda problems, in ‘44th Meeting of the European Working Group “Multicriteria Aid for Decisions”, Brussels, Belgium’.
- Shapley, L. S. (1952), *Notes on the N-Person Game, III: Some Variants of the Von Neumann-Morgenstern Definition of Solution*, RAND Corporation, Santa Monica, CA.
- Shapley, L. S. (1953), A value for n-person games, in H. W. Kuhn & A. W. Tucker, eds, ‘Contributions to the Theory of Games II’, Princeton University Press, Princeton, pp. 307–317.
- Shubik, M. (1962), ‘Incentives, decentralized control, the assignment of joint costs and internal pricing’, *Management Science* **8**(3), 325–343.
URL: <http://www.jstor.org/stable/2627389>
- Simon, H. A. (1962), ‘The architecture of complexity’, *Proceedings of the American Philosophical Society* **106**(6), 467–482.
URL: <http://www.jstor.org/stable/985254>
- Szczepanski, P. L., Barcz, A. S., Michalak, T. P. & Rahwan, T. (2015), The game-theoretic interaction index on social networks with applications to link prediction and community detection, in ‘International Joint Conference on Artificial Intelligence’.
- Tarkowski, M. K., Michalak, T. P. & Wooldridge, M. (2019), ‘A game-theoretic algorithm for link prediction’, *ArXiv* **abs/1912.12846**.
- von Neumann, J. & Morgenstern, O. (1944), *Theory of Games and Economic Behavior*, first edn, princeton.
- Weber, R. J. (1988), *Probabilistic values for games*, Cambridge University Press, p. 101–120.

Apéndice A

Anexo I: Código de Python

A.1. Herramientas

```
1 import heapq
2 from collections import deque, OrderedDict
3 import networkx as nx
4 import math
5
6 def dijkstra(graph):
7     """
8     Función que para cada nodo del grafo o digrafo de la clase Networkx nos almacena
9     una lista de distancias como nuevo atributo. De manera que, cada nodo tendrá a
10    disposición la lista de sus distancias al resto de nodos del grafo graph.
11    Usaremos este método siempre y cuando las aristas del grafo tengan pesos diferentes
12    a 1, ya que en caso contrario se usará la función breadth_first_search.
13
14    Parameters
15    -----
16    graph: Graph or Digraph from Networkx.
17           Grafo que almacena nuestra red de jugadores.
18
19    Returns
20    -----
21    None.
22
23    """
24    for source in graph.nodes:
25        distances= {node: float('inf') for node in graph.nodes}
26        seen_nodes= {node: False for node in graph.nodes}
27        distances[source]= 0
28        priority_queue= [(0, source)]
29        while priority_queue:
30            current_distance, current_node= heapq.heappop(priority_queue)
31            seen_nodes[current_node]= True
32            for neighbor, attributes in graph[current_node].items():
33                weight= attributes.get('weight', 1)
34                if not seen_nodes[neighbor]:
35                    if distances[neighbor]> distances[current_node] + weight:
36                        distances[neighbor]= distances[current_node]+ weight
37                        heapq.heappush(priority_queue, (distances[neighbor], neighbor))
38    graph.nodes[source]['distances']= distances
```

```

39
40
41 def breadth_first_search(graph):
42     """
43     Función que a través del algoritmo de breadth first search busca los caminos mínimos
44     entre cada uno de los nodos de un grafo y los almacena dentro de los atributos de un
45     ↪ nodo
46     como un diccionario. La distancia establecida para aquellos nodos que no están
47     ↪ conectados
48     entre sí será infinito. Este método lo usaremos siempre y cuando el grafo no posea
49     ↪ aristas
50     con pesos, de manera que por defecto el peso de cada una de ellas será 1.
51
52     Parameters
53     -----
54     graph : Graph or Digraph from Networkx
55         Grafo que almacena nuestra red de jugadores.
56
57     Returns
58     -----
59     None.
60
61     """
62     for source in graph.nodes:
63         distances= {node: float('inf') for node in graph.nodes}
64         distances[source]= 0
65         queue= deque([source])
66         while queue:
67             current_node= queue.popleft()
68             for neighbor in graph.neighbors(current_node):
69                 if distances[neighbor]== float('inf'):
70                     distances[neighbor]= distances[current_node]+1
71                     queue.append(neighbor)
72         graph.nodes[source]['distances']= distances
73
74 def compute_shapley(l, n):
75     """
76     Función auxiliar de la función semivalue_interaction_index que me calcula
77     el coeficiente por el cual tenemos que multiplicar la sinergia en el cálculo del
78     índice de interacción del semivalor de Shapley. Este me incluye la probabilidad
79     de que se forme una coalición de tamaño l, de todos los posibles tamaños de
80     ↪ coaliciones del
81     grafo.
82
83     Parameters
84     -----
85     l : int.
86         Entero que me define el tamaño de una posible coalición en la red.
87     n : int.
88         Número de nodos que tiene el grafo.
89
90     Returns
91     -----
92     float
93     Factor por el cual debemos multiplicar la sinergia.

```

```

90
91     """
92     return 1/(math.comb(n-2, 1)*(n-1))
93
94
95 #Funciones auxiliares correspondientes al cálculo de los índices de interacción definidos
96 #por Szczepánski
97
98 def k_neighbors(graph, k, weighted):
99     """
100     Función que crea para cada nodo del grafo una lista con los nodos que se
101     encuentran en su k-vecindad y la almacena como atributo 'k-neighbors'.
102
103     Parameters
104     -----
105     graph : Graph or Digraph from Networkx.
106         Grafo o digrafo que representa nuestra red de jugadores. Este grafo ha de cumplir
107         que para cada nodo exista un diccionario como atributo que almacene la distancia
108         ↪ que hay de dicho nodo
109         a cada uno de sus vecinos.
110     k : int
111         Radio de la vecindad.
112
113     Returns
114     -----
115     None.
116
117     """
118     if weighted:
119         dijkstra(graph)
120     else:
121         breadth_first_search(graph)
122     for node in graph.nodes:
123         graph.nodes[node]['k_neighbors'] = [key for key, value in
124         ↪ graph.nodes[node]['distances'].items() if value <= k]
125         graph.nodes[node]['k_neighbors'].remove(node)
126
127 #Funciones auxiliares correspondientes al cálculo de los índices de interacción definidos
128 ↪ por Tarkowski
129
130 def k_neighbors_distances(graph, k, weighted):
131     """
132     Función que crea para cada nodo del grafo un diccionario el cual contiene como claves
133     aquellos vértices del grafo que se encuentran en la k vecindad del nodo, y como
134     ↪ valores
135     almacenamos la distancia concreta a la cual se encuentran. Fijémonos en que esta
136     ↪ distancia
137     cumplirá ser menor o igual a k. Además este diccionario estará ordenado en orden
138     ↪ descendente
139     de los valores, de manera que el primer nodo del diccionario será aquel que se
140     ↪ encuentra a mayor
141     distancia y el último el más cercano.

```

```

137 El diccionario creado será un atributo propio del nodo llamado
    ↪ "k_neighbors_distances". Es importante destacar
138 que en este caso, a diferencia de en la función k-neighbors para el cálculo del
    ↪ índice de interacción
139 introducido por Szczepánski, no vamos a eliminar la distancia de un nodo a sí mismo,
    ↪ ya que nos será de utilidad
140 en operaciones posteriores.
141
142 Almacenaremos el par (nodo, 0) ya que el output de nuestra función va a ser clave
    ↪ para poder encontrar los conjuntos
143 de nodos que se encuentran a distancias mayores o iguales a una distancia d existente
    ↪ dentro de la k-vecindad. En el
144 caso en que no almacenamos este valor, el primer cálculo de nodos que se encuentran
    ↪ a distancia superior o igual a
145 la máxima existente dentro de la k-vecindad, sumaría un término de más.
146
147 Parameters
148 -----
149 graph : Graph or Digraph from Networkx.
150 Grafo o digrafo que representa nuestra red de jugadores. Este grafo ha de cumplir
151 que para cada nodo exista un diccionario como atributo que almacene la distancia
    ↪ que hay de dicho nodo
152 a cada uno de sus vecinos.
153 k : int
154 Radio de la vecindad.
155 weighted: boolean.
156 Este valor nos indica si nuestro grafo es ponderado o no para saber qué tipo de
    ↪ algoritmo de búsqueda
157 de caminos mínimos poder aplicar.
158
159 Returns
160 -----
161 None.
162
163 """
164 if weighted:
165     dijkstra(graph)
166 else:
167     breadth_first_search(graph)
168 for node in graph.nodes:
169     neighbors= {key: value for key, value in graph.nodes[node]['distances'].items()
    ↪ if value<= k}
170     neighbors= OrderedDict(sorted(neighbors.items(), key= lambda x: x[1], reverse=
    ↪ True)) #usamos un diccionario ordenado
171     graph.nodes[node]['k_neighbors_distances']= neighbors
172
173 def subconjuntos(graph, k, weighted):
174     """
175 Función que para cada nodo del grafo, y para distancia a la que se encuentra al menos
    ↪ uno de los
176 nodos de su k-vecindad, calcula dos cantidades:
177 1. Cuántos nodos de la red se encuentran a una distancia mayor a dicha distancia.
178 2. Cuántos nodos de la red se encuentran a una distancia mayor o igual a dicha
    ↪ distancia.

```

```

179     Estas cantidades las almacenará en un diccionario, el cual para cada distancia,
180     ↪ guarde como valores
181     una lista que almacene las dos cantidades previas. Este diccionario será un atributo
182     ↪ del nodo llamado
183     subsets.
184
185     Parameters
186     -----
187     graph : Grafo o Digrafo de Networkx.
188         Grafo o digrafo que representa nuestra red de jugadores.
189     k: int.
190         Radio de la vecindad.
191     weighted: boolean.
192         Este valor nos indica si nuestro grafo es ponderado o no para saber qué tipo de
193         ↪ algoritmo de búsqueda
194         de caminos mínimos poder aplicar.
195
196     Returns
197     -----
198     None.
199
200     """
201     n= len(graph.nodes)
202     k_neighbors_distances(graph, k, weighted)
203     for node in graph.nodes:
204         subsets_node= {}
205         distances_node= list(graph.nodes[node] ["k_neighbors_distances"].values()) #lista
206         con las distancias ordenadas de mayor a menor dentro de la k-vecindad
207         #para el nodo node
208         current_distance= distances_node[0]
209         nodes_gt_d= n-len(distances_node)
210         nodes_ge_d= nodes_gt_d
211         for dist in distances_node:
212             if dist!= current_distance:
213                 subsets_node[current_distance]= [nodes_gt_d, nodes_ge_d]
214                 nodes_gt_d= nodes_ge_d
215                 current_distance= dist
216                 nodes_ge_d+= 1
217                 subsets_node[current_distance]= [nodes_gt_d, nodes_ge_d]
218         graph.nodes[node] ["subsets"]= subsets_node
219
220 def read_data(dataset_name):
221     """
222     Función utilizada para leer el dataset que define cada una de las redes,
223     introduciendo como parámetro de entrada el nombre de la red que queremos
224     almacenar en el grafo de salida.
225
226     Parameters
227     -----
228     dataset_name : String
229         Nombre del conjunto de datos que vamos a usar para construir nuestra red.
230
231     Returns
232     -----

```

```

231     graph : Graph
232         Grafo que contiene la información del dataset pasado por parámetro
233
234     """
235     file= f"Datasets/{dataset_name}/out_{dataset_name}.txt"
236     graph= nx.read_edgelist(file, nodetype= int, comments= '%')
237     return graph

```

A.2. Problema inicial de predicción de vínculos

Este fichero almacena el código empleado en el experimento de la predicción de vínculos. Para la realización de este, creamos una clase con todas las herramientas necesarias, y presentamos una función que se encarga de computar el estudio con las cinco redes indicadas en la memoria.

```

1  import tools
2  import first_interaction_index
3  from second_interaction_index import interaction_indices_type2
4  import networkx as nx
5  import pandas as pd
6  import matplotlib.pyplot as plt
7  import random
8  import statistics
9  import numpy as np
10 import heapq
11 from collections import OrderedDict
12 import ipdb
13 import time
14
15
16 class lp_first_experiment():
17     def __init__(self, graph, k_index, weighted, similarity_measure):
18         """
19         Este método de inicialización de un experimento almacena los datos necesarios
20         ↪ para proceder
21         a realizar el estudio del impacto del índice de interacción de Shapley en
22         ↪ problemas de predicción
23         de vínculos.
24
25         Parameters
26         -----
27         graph : Graph o Digraph de Networkx.
28             Red de jugadores para la cual vamos a realizar el experimento.
29         weighted : Boolean
30             Valor booleano que nos indica si las aristas del grafo tienen peso o no.
31         k_index: int
32             Entero que nos indica el radio de las localidades de vecinos que queremos
33             ↪ considerar.
34         similarity_measure: String.
35             Cadena de texto que me indica el índice de similitud que quiero utilizar para
36             ↪ hacer el estudio
37             correspondiente de predicción de vínculos. Dentro de los posibles índices de
38             ↪ similitud tenemos:
39             - GSVII: General Shapley Value Interaction Index.
40             - GBVII: General Banzhaf Value Interaction Index.

```

```

36         - SVII: Shapley Value Interaction Index (Szczepánski)
37         - GSVMI: General Shapley Value Modified Interaction Index.
38         - GBVMI: General Banzhaf Value Modified Interaction Index.
39         - SVMII: Shapley Value Modified Interaction Index (Tarkowski).
40         - CNN: Common Neighbors.
41         - AAI: Adamic Adar Index.
42
43     Returns
44     -----
45     None.
46
47     """
48     self.graph= graph
49     self.all_possible_edges= {(u,v) for u in self.graph.nodes for v in range(1, u)}
50     ↪ #conjunto de aristas que tendría el grafo completo con los nodos del nuestro
51     self.weighted= weighted
52     self.k_index= k_index
53     self.similarity_measure= similarity_measure
54
55 def compute_similarities(self, graph_removed):
56     """
57     Método de la clase lp_first_experiment que computa el índice de interacción que
58     ↪ queremos estudiar en el
59     experimento para cada par de nodos no emparejados en el grafo de entrenamiento
60     ↪ pasado como parámetro de entrada.
61     Para ello hace un estudio previo de quiénes son los nodos que no poseen arista
62     ↪ entre ellos y hace llamamiento a las
63     precomputaciones necesarias para cada uno de los algoritmos de similitud.
64
65     Parameters
66     -----
67     graph_removed : TYPE
68         DESCRIPTION.
69
70     Returns
71     -----
72     ranking: Ordered Dictionary.
73     Diccionario ordenado que surge de haber reordenado los pares (arista, índice)
74     ↪ en orden ascendente del índice.
75
76     """
77     existent_links= {(u, v) if u> v else (v, u) for u, v in graph_removed.edges}
78     non_existent_links= list(self.all_possible_edges - existent_links)
79     ranking= {}
80     if self.similarity_measure== 'CNN':
81         tools.k_neighbors(graph_removed, self.k_index, self.weighted)
82         for edge in non_existent_links:
83             ranking[edge]=
84                 ↪ first_interaction_index.common_neighbors_similarity(graph_removed,
85                 ↪ edge[0], edge[1])
86     elif self.similarity_measure== 'AAI':
87         tools.k_neighbors(graph_removed, self.k_index, self.weighted)
88         for edge in non_existent_links:
89             ranking[edge]= first_interaction_index.adamic_adar_index(graph_removed,
90             ↪ edge[0], edge[1])

```

```

83     elif self.similarity_measure== 'SVII':
84         tools.k_neighbors(graph_removed, self.k_index, self.weighted)
85         for edge in non_existent_links:
86             ranking[edge]=
87                 ↪ first_interaction_index.shapley_interaction_index(graph_removed,
88                 ↪ edge[0], edge[1])
89     elif self.similarity_measure== 'SVMII':
90         tools.subconjuntos(graph_removed, self.k_index, self.weighted)
91         for edge in non_existent_links:
92             ranking[edge]=
93                 ↪ first_interaction_index.tarkowski_shapley_II(graph_removed, edge[0],
94                 ↪ edge[1])
95     elif self.similarity_measure== 'GSVII':
96         tools.k_neighbors(graph_removed, self.k_index, self.weighted) #primero se
97         ↪ realizan los cálculos de distancias sobre el grafo
98         element= interaction_indices_type2(graph= graph_removed, k_index=
99         ↪ self.k_index, weighted= self.weighted, semivalue= "Shapley") #y después
100        ↪ se crea una clase con el grafo como atributo
101        for edge in non_existent_links:
102            ranking[edge]= element.semivalue_interaction_index(edge[0], edge[1])
103    elif self.similarity_measure== 'GBVII':
104        tools.k_neighbors(graph_removed, self.k_index, self.weighted)
105        element= interaction_indices_type2(graph= graph_removed, k_index=
106        ↪ self.k_index, weighted= self.weighted, semivalue= "Banzhaf")
107        for edge in non_existent_links:
108            ranking[edge]= element.semivalue_interaction_index(edge[0], edge[1])
109    elif self.similarity_measure== 'GSVMII':
110        tools.subconjuntos(graph_removed, self.k_index, self.weighted)
111        element= interaction_indices_type2(graph= graph_removed, k_index=
112        ↪ self.k_index, weighted= self.weighted, semivalue= "Shapley")
113        clasement= element.tarkowski_semivalue_II()
114        for edge in non_existent_links:
115            ranking[edge]= clasement[edge[0]-1][edge[1]-1]
116    elif self.similarity_measure== 'GBVMII':
117        tools.subconjuntos(graph_removed, self.k_index, self.weighted)
118        element= interaction_indices_type2(graph= graph_removed, k_index=
119        ↪ self.k_index, weighted= self.weighted, semivalue= "Banzhaf")
120        clasement= element.tarkowski_semivalue_II()
121        for edge in non_existent_links:
122            ranking[edge]= clasement[edge[0]-1][edge[1]-1]
123    else:
124        raise ValueError("Medida de similitud no soportada elija alguna de las
125        ↪ siguientes: CNN, SVII, SVMII, GSVII, GBVII, GSVMII, GBVMII")
126    return OrderedDict(sorted(ranking.items(), key= lambda x: x[1]))
127
128 def precision(self, graph_removed, num_missing_links, ranking):
129     """
130     Método que calcula la precisión total del índice de interacción de Shapley.
131     El método empleado es tomar el número de aristas eliminadas del grafo:
132     ↪ missing_links y
133     tomar missing_links primeras aristas rankeadas según el índice de interacción.
134     ↪ Comprobamos
135     si estas aristas son efectivamente de las "faltantes" o del grupo de las "no
136     ↪ existentes".
137     La métrica final será predichas_faltantes/missing_links.

```

```

124
125 Parameters
126 -----
127 graph_removed : Graph or Digraph from Networkx.
128     Grafo de entrenamiento al que le hemos quitado un porcentaje de sus aristas.
129 num_missing_links: int
130     Nos aporta el número de aristas que han sido eliminadas del grafo para el
131     ↪ entrenamiento.
132 ranking: Ordered Dictionary.
133     Diccionario ordenado de la librería collections que contiene pares de nodos
134     ↪ sin conectar en el
135     grafo de entrenamiento, ordenados de manera ascendente en función de su
136     ↪ índice de interacción.
137
138 Returns
139 -----
140 precision: int.
141     precision*1000.
142
143 """
144 top_links= list(ranking.keys())[: num_missing_links] #[key for _, key in
145     ↪ n_smallest]
146 precision= 0
147 for link in top_links:
148     precision+= 1 if self.graph.has_edge(link[0], link[1]) else 0
149 return (precision/num_missing_links)*1000
150
151 def area_under_curve(self, graph_removed, edges_to_remove, ranking):
152     """
153     Método para el cálculo del área bajo la curva.
154
155     Parameters
156     -----
157     graph_removed : Graph or Digraph from Networkx
158         Grafo o digrafo al cual le hemos quitado un porcentaje de sus aristas.
159     edges_to_remove: list
160         Lista de aristas eliminados del grafo original, es decir, missing_links.
161     ranking: Ordered Dictionary.
162         Diccionario ordenado de la librería collections que contiene pares de nodos
163         ↪ sin conectar en el
164         grafo de entrenamiento, ordenados de manera ascendente en función de su
165         ↪ índice de interacción.
166
167     Returns
168     -----
169     AUC: float
170
171     """
172     right, wrong= 0,0
173     missing_links= [(u, v) if u> v else (v, u) for u, v in edges_to_remove]
174     existent_links= {(u, v) if u> v else (v, u) for u, v in self.graph.edges}
175     non_existent_links= list(self.all_possible_edges - existent_links)
176     for m in missing_links:
177         for l in non_existent_links:

```

```

172         if ranking[m]< ranking[l]: #si el valor asignado a m es menor que el
173             ↪ asignado a l, es más recomendado
174             right+=1
175         elif ranking[m] == ranking[l]: #en caso contrario es igual o menos
176             ↪ recomendado el missing_link que el non_existent
177             wrong+=1
178     total= len(missing_links)*len(non_existent_links)
179     return ((right+ wrong/2)/total)*1000
180
181 def metricas_rendimiento(self, percentage= 0.4):
182     """
183     Método de la clase experimento que usaremos para evaluar las métricas
184     de precisión y area under the curve, insertando el porcentaje de aristas
185     que queremos eliminar de manera aleatoria para la evaluación del índice de
186     interacción del valor de Shapley.
187
188     Parameters
189     -----
190     percentage : float, optional
191         Número perteneciente al intervalo [0,1] que indica el porcentaje de aristas a
192         ↪ eliminar
193         para el estudio. El valor por defecto es 0.4.
194
195     Returns
196     -----
197     (precision, AUC): tuple
198         Tupla con las dos métricas.
199
200     """
201     graph_removed= self.graph.copy()
202     num_edges_remove= round(len(self.graph.edges)* percentage) #he cambiado la manera
203     ↪ de coger el número de aristas a eliminar: antes int
204     edges_to_remove= random.sample(list(self.graph.edges), num_edges_remove)
205     graph_removed.remove_edges_from(edges_to_remove)
206     ranking= self.compute_similarities(graph_removed)
207     precision= self.precision(graph_removed, len(edges_to_remove), ranking)
208     AUC= self.area_under_curve(graph_removed, edges_to_remove, ranking)
209     return (precision, AUC)
210
211 def study(k_index= 2, percentage= 0.3, similarity_measure= 'SVII'):
212     """
213     Método que nos hace un estudio de las métricas de rendimiento del algoritmo de
214     ↪ cálculo
215     del índice de interacción del valor de Shapley. Crea un experimento por dataset a
216     analizar y, tras insertar el valor del parámetro k nos devuelve el promedio de los
217     ↪ valores de
218     las métricas obtenidos para 30 iteraciones.
219
220     Parameters
221     -----
222     k_index : int, optional
223         El valor del radio de la localidad tomada. El valor por defecto es 2.

```

```

221     Returns
222     -----
223     results : Dataframe de pandas.
224     Nos devuelve los promedios de las métricas para cada dataset.
225
226     """
227     beginning= time.perf_counter()
228     graph_zachary= tools.read_data("zachary")
229     graph_dolphins= tools.read_data("dolphins")
230     graph_polbooks= tools.read_data("polbooks")
231     graph_football= tools.read_data("football")
232     graph_jazz= tools.read_data("jazz")
233     zachary_experiment= lp_first_experiment(graph= graph_zachary, k_index= k_index,
234     ↪ weighted= False, similarity_measure= similarity_measure)
235     dolphins_experiment= lp_first_experiment(graph= graph_dolphins, k_index= k_index,
236     ↪ weighted= False, similarity_measure= similarity_measure)
237     polbooks_experiment= lp_first_experiment(graph= graph_polbooks, k_index= k_index,
238     ↪ weighted= False, similarity_measure= similarity_measure)
239     football_experiment= lp_first_experiment(graph= graph_football, k_index= k_index,
240     ↪ weighted= False, similarity_measure= similarity_measure)
241     jazz_experiment= lp_first_experiment(graph= graph_jazz, k_index= k_index, weighted=
242     ↪ False, similarity_measure= similarity_measure)
243     zachary_precision, zachary_auc, dolphins_precision, dolphins_auc, polbooks_precision,
244     ↪ polbooks_auc, football_precision, football_auc, jazz_precision, jazz_auc=
245     ↪ np.zeros(100), np.zeros(100), np.zeros(100), np.zeros(100), np.zeros(100),
246     ↪ np.zeros(100), np.zeros(100), np.zeros(100), np.zeros(100), np.zeros(100)
247     for i in range(100):
248         zachary_precision[i], zachary_auc[i]=
249         ↪ zachary_experiment.metricas_rendimiento(percentage)
250         dolphins_precision[i], dolphins_auc[i]=
251         ↪ dolphins_experiment.metricas_rendimiento(percentage)
252         polbooks_precision[i], polbooks_auc[i]=
253         ↪ polbooks_experiment.metricas_rendimiento(percentage)
254         football_precision[i], football_auc[i]=
255         ↪ football_experiment.metricas_rendimiento(percentage)
256         jazz_precision[i], jazz_auc[i]= jazz_experiment.metricas_rendimiento(percentage)
257     mean_precision= [np.mean(zachary_precision), np.mean(dolphins_precision),
258     ↪ np.mean(polbooks_precision), np.mean(football_precision),
259     ↪ np.mean(jazz_precision)]
260     mean_auc= [np.mean(zachary_auc), np.mean(dolphins_auc), np.mean(polbooks_auc),
261     ↪ np.mean(football_auc), np.mean(jazz_auc)]
262     results= pd.DataFrame({
263         "Dataset": ['Zachary', 'Dolphins', 'Polbooks', 'Football', 'Jazz'],
264         "Precision": mean_precision,
265         "AUC": mean_auc})
266     end= time.perf_counter()
267     execution_time= end-beginning
268     print(f"The execution time was {execution_time:.5f} segundos")
269     return results

```

En este fichero almacenamos las funciones que nos permiten el cálculo de las medidas de similitud: k-common neighbors, adamic-adar index, índice de interacción de k-grado de Shapley (SVII), índice de interacción de k-pasos amortiguado (SVMII). A toda medida de similitud le corresponden dos funciones de cálculo, aquella que se llama *extended* es la función que nos calcula a la vez las similitudes entre todos los vértices del grafo. Mientras que, la otra calcula la similitud entre los

vértices insertados.

```
1
2 import tools
3 import networkx as nx
4 from math import log
5 import numpy as np
6
7
8 def common_neighbors_similarity(graph, s, t):
9     """
10     Función que calcula el número de vecinos comunes dentro de una k-vecindad para el par
11     ↪ de nodos
12     del grafo insertado. El resultado será su valor opuesto, es decir, multiplicándolo
13     ↪ por menos uno,
14     ya que a la hora de evaluar cuál se recomienda más, los índices de interacción
15     ↪ calculados siguen ese criterio.
16
17     Previamente a llamar a esta función ya se ha hecho el cálculo de las k-vecindades de
18     ↪ los nodos del grafo.
19
20     Parameters
21     -----
22     graph : Graph from Networkx.
23         Grafo o digrafo que representa la red de agentes, el cual debe tener almacenado
24         ↪ para cada nodo un atributo
25         que represente su k-localidad llamado k_neighbors. Además los nodos están
26         ↪ representados a través de números
27         enteros.
28     s, t: int.
29         Nodos cuyo índice de interacción queremos calcular.
30
31     Returns
32     -----
33     common_neighbors: int.
34         Número de vecinos comunes en las k-vecindades del nodo s y t.
35     """
36     common_neighbors=
37     ↪ -len(set(graph.nodes[s]['k_neighbors'])&set(graph.nodes[t]['k_neighbors']))
38     return common_neighbors
39
40 def common_neighbors_extended(graph):
41     """
42     Función que calcula la similitud en términos de número de vecinos comunes dentro de
43     ↪ una vecindad de radio k. El conjunto
44     de los vecinos dentro de la k-vecindad para cada uno de los nodos es computado
45     ↪ previamente a través de la función k_neighbors
46     del script tools.
47
48     Parameters
49     -----
50     graph : Graph from Networkx.
51         Grafo o digrafo que representa la red de agentes, el cual debe tener almacenado
52         ↪ para cada nodo un atributo que represente
53         su k-localidad llamado k_neighbors.
54
55     """
```

```

45     Returns
46     -----
47     common_neighbors : squared numpy array.
48         Matriz de numpy triangular inferior con ceros en la diagonal que almacena para
49         ↪ cada
50         par de nodos el índice de interacción entre ellos, se accede a él tomando el nodo
51         con índice más mayor como fila y el de menor índice como columna.
52
53     """
54     common_neighbors= np.zeros((len(graph.nodes), len(graph.nodes)), dtype= float)
55     for s in sorted(graph.nodes, reverse= True):
56         for t in range(1, s):
57             common_neighbors[s-1][t-1]=
58                 ↪ -len(set(graph.nodes[s]['k_neighbors'])&set(graph.nodes[t]['k_neighbors']))
59     return common_neighbors
60
61 def adamic_adar_index(graph, s , t):
62     """
63     Función que calcula el índice de similitud de Adamic Adar entre dos nodos de un
64     ↪ grafo.
65
66     Parameters
67     -----
68     graph: Graph from Networkx.
69         Grafo o digrafo que representa la red de agentes, el cual debe tener para cada
70         ↪ nodo un atributo que represente su k-localidad
71         llamado k-neighbors.
72     s, t: int.
73         Nodos del grafo graph.
74
75     Returns
76     -----
77     adamic_adar_index: float.
78         Índice de similitud de Adamic Adar entre los nodos s, t, opuesto en signo para
79         ↪ que siga la misma lógica que los índices de interacción.
80     """
81     adamic_adar_index=0
82     intersection= set(graph.nodes[s]['k_neighbors'])&set(graph.nodes[t]['k_neighbors'])
83     for node in intersection:
84         adamic_adar_index-= 1/(log(len(graph.nodes[node]['k_neighbors'])))
85     return adamic_adar_index
86
87 def adamic_adar_extended(graph):
88     """
89     Función que calcula el índice de similitud de Adamic Adar entre todo par de nodos del
90     ↪ grafo, el índice de similitud, a pesar
91     de que en realidad es positivo, se da su valor opuesto, para poder seguir la misma
92     ↪ lógica en los experimentos que con los índices
93     de interacción.
94
95     Parameters
96     -----
97     graph : Graph from Networkx.
98         Grafo de estudio.

```

```

93     Returns
94     -----
95     adamic_adar : squared numpy array.
96         Matriz de numpy triangular inferior con ceros en la diagonal que almacena para
97         ↪ cada
98         par de nodos el índice de similitud AAI entre ellos.
99
100     """
101     adamic_adar= np.zeros((len(graph.nodes), len(graph.nodes)), dtype= float)
102     for s in sorted(graph.nodes, reverse= True):
103         for t in range(1, s):
104             index, intersection= 0,
105             ↪ set(graph.nodes[s]['k_neighbors'])&set(graph.nodes[t]['k_neighbors'])
106             for node in intersection:
107                 index+= 1/(log(len(graph.nodes[node]['k_neighbors'])))
108             adamic_adar[s-1][t-1]= index
109     return adamic_adar
110
111 def shapley_interaction_index(graph, s, t):
112     """
113     Función que calcula el índice de interacción de Shapley para el par de nodos
114     ↪ insertado como parámetros
115     de la función. Dado que para hacer los cálculos del índice de interacción entre dos
116     ↪ nodos
117     cualesquiera del grafo, es necesario contar con la información de qué nodos se
118     ↪ encuentran en la k-vecindad
119     de cada uno de ellos, previamente a llamar a esta función se ha hecho uso de la
120     ↪ función auxiliar
121     k-neighbors.
122
123     Parameters
124     -----
125     graph : Graph from Networkx.
126         Grafo o digrafo que representa la red de agentes, el cual debe tener almacenado
127         ↪ para cada nodo un atributo
128         que represente su k-localidad llamado k_neighbors. Además los nodos están
129         ↪ representados a través de números
130         enteros.
131     s, t: int.
132         Nodos cuyo índice de interacción queremos calcular.
133
134     Returns
135     -----
136     interaction_index : float.
137         Número flotante negativo o cero que indica el índice de interacción de k-grado de
138         Shapley. Cuánto menor sea el valor del índice de interacción, más similares serán
139         ↪ los nodos.
140
141     """
142     interaction_index= 0
143     for u in set(graph.nodes[s]['k_neighbors'])&set(graph.nodes[t]['k_neighbors']):
144         interaction_index+= 1/(len(graph.nodes[u]['k_neighbors'])-1)
145     if s in graph.nodes[t]['k_neighbors']:
146         if len(graph.nodes[t]['k_neighbors'])>1:

```

```

139         interaction_index -= 1/(len(graph.nodes[t]['k_neighbors'])-1)
140     if len(graph.nodes[s]['k_neighbors']) > 1:
141         interaction_index -= 1/(len(graph.nodes[s]['k_neighbors'])-1)
142     return interaction_index
143
144 def shapley_II_extended(graph):
145     """
146     Función que calcula el índice de interacción de k-grado de Shapley para cada par de
147     ↪ nodos posible
148     en un grafo, devolviendo así una matriz triangular estrictamente inferior.
149
150     Parameters
151     -----
152     graph : Graph from Networkx.
153         Grafo o digrafo que representa la red de agentes, el cual debe tener almacenado
154         ↪ para cada nodo un atributo
155         que represente su k-localidad llamado k_neighbors. Además los nodos están
156         ↪ representados a través de números
157         enteros.
158
159     Returns
160     -----
161     interaction_index : squared numpy array.
162         Matriz de numpy triangular inferior con ceros en la diagonal que almacena para
163         ↪ cada
164         par de nodos el índice de interacción entre ellos, se accede a él tomando el nodo
165         con índice más mayor como fila y el de menor índice como columna.
166     """
167     interaction_index = np.zeros((len(graph.nodes), len(graph.nodes)), dtype= float)
168     for s in sorted(graph.nodes, reverse= True): #ya me los ordena aquí en función de las
169     ↪ etiquetas de los nodos n,..., 1
170         for t in range(1, s):
171             index = 0
172             for u in
173                 ↪ set(graph.nodes[s]['k_neighbors']) & set(graph.nodes[t]['k_neighbors']):
174                     index -= 1/(len(graph.nodes[u]['k_neighbors'])-1)
175             if s in graph.nodes[t]['k_neighbors']:
176                 if len(graph.nodes[s]['k_neighbors']) > 1:
177                     index -= 1/(len(graph.nodes[s]['k_neighbors'])-1)
178                 if len(graph.nodes[t]['k_neighbors']) > 1:
179                     index -= 1/(len(graph.nodes[t]['k_neighbors'])-1)
180             interaction_index[s-1][t-1] = index
181     return interaction_index
182
183 def tarkowski_shapley_II(graph, s, t):
184     """
185     Función de cálculo del índice de interacción definido por Tarkowski para el semivalor
186     ↪ de
187     Shapley. Este cálculo no tiene en consideración el tamaño de las posibles
188     ↪ coaliciones, es decir,
189     el parámetro l, luego su cálculo va a ser mucho más simple.
190
191     En esta función aprovecharemos el cálculo hecho en la función subconjuntos del script
192     ↪ de tools

```

```

184 y, simplemente para calcular los nodos que se encuentran a distancia menor o menor o
185 ↪ igual que otra
186 de un nodo, le restaremos al total de nodos las cantidades correspondientes
187 ↪ calculadas en subconjuntos.
188 Puesto que sería ineficiente llamar a la función subconjuntos cada vez que se ejecute
189 ↪ el índice de inte
190 racción entre dos vértices, este será un paso previo a utilizar esta función.
191
192 Parameters
193 -----
194 graph : Graph from Networkx.
195     Grafo o digrafo que representa la red de agentes, el cual debe tener almacenado
196     ↪ para cada nodo un atributo
197     que represente su k-localidad llamado k_neighbors. Además los nodos están
198     ↪ representados a través de números
199     enteros.
200 s, t: int.
201     Nodos cuyo índice de interacción queremos calcular.
202
203 Returns
204 -----
205 interaction_index: float.
206     Valor del índice de interacción de tarkowski entre los nodos s, t del grafo.
207 """
208
209 n, nodes= len(graph.nodes), graph.nodes
210 interaction_index= 0
211 for u in nodes:
212     minimo= min(nodes[u]["k_neighbors_distances"].get(s, -1),
213                 ↪ nodes[u]["k_neighbors_distances"].get(t, -1))
214     maximo= max(nodes[u]["k_neighbors_distances"].get(s, -1),
215                ↪ nodes[u]["k_neighbors_distances"].get(t, -1))
216     higher_distances= {dist for dist in nodes[u]["k_neighbors_distances"].values() if
217                        ↪ dist> maximo}
218     #de la vecindad k de u, y entonces calcularemos el número de posibles coaliciones
219     ↪ a esa distancia.
220     if minimo!= -1: #ninguno de los dos se encuentra fuera de la k-vecindad, ambos no
221     ↪ pueden ser el mismo nodo luego d!=0 siempre
222     nodos_le= n- nodes[u]["subsets"][maximo][0]
223     positive_mc= 0 if nodos_le==1 else 1/((maximo**2)*(nodos_le -1))
224     negative_mc= 0
225     if higher_distances:
226         for dist in higher_distances:
227             nodos_le_dist= n- nodes[u]["subsets"][dist][0]
228             nodos_lt_dist= n- nodes[u]["subsets"][dist][1]
229             negative_mc += 0 if (nodos_le_dist==1 or nodos_lt_dist==1) else
230             ↪ (1/(dist**2))*(1/(nodos_lt_dist -1) -1/(nodos_le_dist -1)) #mirar
231             ↪ si puede existir el caso de denominador negativo
232         interaction_index+= (negative_mc - positive_mc)
233 return interaction_index
234
235 def tarkowski_shapley_II_extended(graph):
236     """
237     Función de cálculo del índice de interacción definido por Tarkowski para el semivalor
238     ↪ de

```

```

226     Shapley. Esta hace exactamente lo mismo que la anterior, pero en lugar de computar el
227     ↪ índice
228
229     de interacción para el par de nodos indicado, lo calcula para todos los nodos del
230     ↪ grafo.
231
232     Parameters
233     -----
234
235     graph : Graph from Networkx.
236     Grafo o digrafo que representa la red de agentes, el cual debe tener almacenado
237     ↪ para cada nodo un atributo
238     que represente su k-localidad llamado k_neighbors. Además los nodos están
239     ↪ representados a través de números
240     enteros.
241
242     Returns
243     -----
244
245     interaction_index: squared numpy array.
246     Matriz de numpy triangular inferior con ceros en la diagonal que almacena para
247     ↪ cada
248     par de nodos el índice de interacción entre ellos, se accede a él tomando el nodo
249     con índice más mayor como fila y el de menor índice como columna.
250
251     """
252     n= len(graph.nodes)
253     interaction_index= np.zeros((n, n), dtype= float)
254     for u in graph.nodes:
255         distances_items= list(graph.nodes[u]["k_neighbors_distances"].items())
256         subsets= graph.nodes[u]["subsets"]
257         higher_distance= distances_items[0][1]
258         negative_mc= 0
259         while len(distances_items)> 1:
260             s, d= distances_items.pop(0)
261             if d!= higher_distance:
262                 nodos_lt_hd= n - subsets[higher_distance][1]
263                 nodos_le_hd= n - subsets[higher_distance][0]
264                 negative_mc+= 0 if (nodos_lt_hd==1 or nodos_le_hd==1) else
265                 ↪ (1/(higher_distance**2))*(1/(nodos_lt_hd -1) -1/(nodos_le_hd -1))
266                 higher_distance= d
267             for (t, dt) in distances_items:
268                 nodos_le= n- subsets[d][0]
269                 positive_mc= 0 if nodos_le==1 else 1/((d**2)*(nodos_le -1))
270                 if s> t:
271                     interaction_index[s-1][t-1]+= negative_mc - positive_mc
272                 else:
273                     interaction_index[t-1][s-1]+= negative_mc - positive_mc
274     return interaction_index
275
276 import networkx as nx
277 import tools
278 import matplotlib.pyplot as plt
279 import math
280 import numpy as np
281 import heapq
282 from collections import OrderedDict
283

```

```

275 class interaction_indices_type2():
276
277     def __init__(self, graph, k_index, weighted, semivalue):
278         """
279         Este método de inicialización de un experimento almacena los datos necesarios
280         ↪ para proceder
281         a realizar el estudio del impacto del índice de interacción de Shapley en
282         ↪ problemas de predicción
283         de vínculos.
284
285         Parameters
286         -----
287         graph : Graph o Digraph de Networkx.
288             Red de jugadores para la cual vamos a realizar el experimento.
289         weighted : Boolean
290             Valor booleano que nos indica si las aristas del grafo tienen peso o no.
291         k_index: int
292             Entero que nos indica el radio de las localidades de vecinos que queremos
293             ↪ considerar.
294         similarity_measure: String.
295             Cadena de texto que me indica el índice de similitud que quiero utilizar para
296             ↪ hacer e
297
298         Returns
299         -----
300         None.
301
302         """
303         self.graph= graph
304         self.n= len(graph.nodes)
305         self.weighted= weighted
306         self.k_index= k_index
307         self.semivalue = semivalue
308         if semivalue == "Shapley":
309             self.l_factors= [tools.compute_shapley(l, self.n) for l in range(0, self.n
310             ↪ -1)]
311         else:
312             self.l_factors= None
313
314     def tarkowski_semivalue_II(self):
315         """
316         Función que me calcula el índice de interacción del semivalor definido
317         en el artículo de Tarkowski. Para el cálculo de la función de distribución de
318         probabilidad discreta, puesto que se repite para cada par de nodos s, t su
319         ↪ cálculo,
320         hemos decidido englobar el cálculo del factor correspondiente incluyendo el
321         ↪ número
322         combinatorio ( $|V|-2$ ) sobre l, fuera de la función, y que este sea un parámetro de
323         ↪ entrada
324         para el caso en el que se elija el semivalor de Shapley. Puesto que en el caso
325         ↪ del
326         semivalor de Banzhaf, es cierto que la expresión se reduce a la división por la
327         constante  $2*(n-2)$ .

```

```

321
322 Raises
323 -----
324 ValueError
325     En el caso de que se quiera calcular un índice de interacción procedente de
326     un semivalor distinto al de Shapley o el de Banzhaf, se levanta una
327     ↪ excepción.
328
329 Returns
330 -----
331 interaction_index: squared numpy array
332     Matriz cuadrada triangular inferior con los índices de interacción entre cada
333     ↪ par de nodos
334     diferentes.
335 """
336 #tools.subconjuntos(self.graph, self.k_index, self.weighted)
337 n, nodes= self.n, self.graph.nodes
338 interaction_index= np.zeros((n, n), dtype= float)
339 if self.semivalue == "Shapley":
340     for u in nodes:
341         subsets= self.graph.nodes[u]["subsets"]
342         for l in range(n-1):
343             distances_items= list(nodes[u]["k_neighbors_distances"].items())
344             higher_distance= distances_items[0][1]
345             negative_l= 0
346             while len(distances_items)>1:
347                 s, d= distances_items.pop(0)
348                 if d!= higher_distance:
349                     nodos_ge_h= subsets[higher_distance][0]
350                     nodos_gt_h= subsets[higher_distance][1]
351                     negative_l+= (self.l_factors[l]/
352                                 ↪ higher_distance**2)*(math.comb(nodos_ge_h, l)-
353                                 ↪ math.comb(nodos_gt_h, l))
354                     higher_distance= d
355             for (t, dt) in distances_items:
356                 nodos_gt= subsets[d][1]
357                 positive_l= (self.l_factors[l]/d**2)*math.comb(nodos_gt, l)
358                 if s>t:
359                     interaction_index[s-1][t-1]+= negative_l- positive_l
360                 else:
361                     interaction_index[t-1][s-1]+= negative_l- positive_l
362 elif self.semivalue == "Banzhaf":
363     for u in nodes:
364         subsets= self.graph.nodes[u]["subsets"]
365         for l in range(n-1):
366             distances_items= list(nodes[u]["k_neighbors_distances"].items())
367             higher_distance= distances_items[0][1]
368             negative_l= 0
369             while len(distances_items)>1:
370                 s, d= distances_items.pop(0)
371                 if d!= higher_distance:
372                     nodos_ge_h= subsets[higher_distance][0]
373                     nodos_gt_h= subsets[higher_distance][1]
374                     negative_l+= (1/higher_distance**2)*(math.comb(nodos_ge_h,
375                                 ↪ l)- math.comb(nodos_gt_h, l))

```

```

371         higher_distance= d
372     for (t, dt) in distances_items:
373         nodos_gt= subsets[d][1]
374         positive_l= (1/d**2)*math.comb(nodos_gt, 1)
375         if s>t:
376             interaction_index[s-1][t-1]+= (negative_l-
377                 ↪ positive_l)/(2**(n-2))
378         else:
379             interaction_index[t-1][s-1]+= (negative_l-
380                 ↪ positive_l)/(2**(n-2))
381     else:
382         raise ValueError("Semivalor no soportado elija bien Shapley o Banzhaf")
383     return interaction_index
384
385 def semivalue_interaction_index(self, s, t):
386     """
387     Función que me calcula el índice de interacción de k-grado de Shapley para cada
388     ↪ par de
389     nodos del grafo. Esta función a raíz de un grafo y la especificación del
390     ↪ semivalor del cual
391     deriva nuestro índice de interacción, hará uso de la función auxiliar
392     ↪ compute_shapley y
393     del factor constante de Banzhaf para calcular el factor el cual me incluye la
394     ↪ función de la distribución de probabilidad
395     discreta correspondiente.
396
397     Parameters
398     -----
399     s, t: int.
400         Nodos cuyo índice de interacción queremos calcular.
401
402     Raises
403     -----
404     ValueError
405         En el caso de que se quiera calcular un índice de interacción procedente de
406         un semivalor distinto al de Shapley o el de Banzhaf, se levanta una
407         ↪ excepción.
408
409     Returns
410     -----
411     interaction_index : float.
412         Valor del índice de interacción de Shapley reducido de Szczepánski entre los
413         ↪ nodos s, t.
414
415     """
416     #tools.k_neighbors(graph, k , weighted)
417     n, nodes= self.n, self.graph.nodes
418     interaction_index= 0
419     if self.semivalue == "Shapley":
420         for l in range(n -1):
421             synergy= 0
422             for w in set(nodes[s]['k_neighbors']&set(nodes[t]['k_neighbors'])):
423                 synergy+= math.comb(n -1-len(nodes[w]['k_neighbors']), 1)
424             if s in nodes[t]['k_neighbors']:
425                 synergy+= math.comb(n-1-len(nodes[t]['k_neighbors']), 1)

```

```

418         synergy-= math.comb(n-1-len(nodes[s]['k_neighbors']), 1)
419         interaction_index+= self.l_factors[l]*synergy
420     elif self.semivalue == "Banzhaf":
421         for l in range(n-1):
422             synergy= 0
423             for w in set(nodes[s]['k_neighbors'])&set(nodes[t]['k_neighbors']):
424                 synergy-= math.comb(n-1-len(nodes[w]['k_neighbors']), 1)
425             if s in nodes[t]['k_neighbors']:
426                 synergy-= math.comb(n-1-len(nodes[t]['k_neighbors']), 1)
427                 synergy-= math.comb(n-1-len(nodes[s]['k_neighbors']), 1)
428             interaction_index+= synergy
429         interaction_index/= (2**(n-2)) #el factor es igual para cualquier valor de l
430     else:
431         raise ValueError("Semivalor no soportado elija bien Shapley o Banzhaf")
432     return interaction_index
433

```

A.3. Problema de reclasificación

Script que realiza el experimento de reclasificación.

```

1  import networkx as nx
2  import numpy as np
3  import pandas as pd
4  import math
5  import random
6  import tools
7  from collections import OrderedDict
8  import first_interaction_index
9  import second_interaction_index
10 import link_prediction
11 import time
12
13
14 class lp_second_experiment():
15
16     def __init__(self, graph, k_index, number_remove, batches_size, similarity_measure,
17         ↪ method= 'score', weighted= False):
18         """
19         Método de inicialización del experimento de rerankeo. Este nos sirve para poner
20         ↪ en común todas las características
21         que comparten las diferentes creaciones de experimentos. Además de los valores
22         ↪ insertados por parámetros se crearán ciertas
23         variables necesarias como el número de rankings generados, una lista que almacena
24         ↪ cada uno de los rankings, una lista que nos
25         indica las aristas que han sido insertadas en el proceso, etc.
26
27         Parameters
28         -----
29         graph: Networkx graph.
30             Grafo sobre el que vamos a realizar el experimento.
31         k_index: int.
32             Número entero que nos definirá el radio para aplicar nuestra medida de
33             ↪ similitud.
34         number_remove: int.

```

```

30         Número de aristas a quitar del grafo para realizar la comparación.
31     batches_size: int.
32         Entero el cual nos define el tamaño de los lotes de aristas que insertaremos
33         ↪ paso a paso en el grafo.
34     similarity_measure: String.
35         Cadena de texto que nos indica la medida de similitud que emplearemos para
36         ↪ realizar el experimento.
37     method: String.
38         Cadena que nos define el criterio seguido para insertar aristas en el grafo:
39         ↪ orden en el ranking o aleatorio.
40     weighted: Boolean value.
41         Valor booleano que nos dice si el grafo es ponderado o no.
42 """
43 self.graph= graph
44 self.k_index= k_index
45 self.weighted= weighted
46 self.number_remove= number_remove
47 self.batches_size= batches_size
48 self.similarity_measure= similarity_measure
49 self.method= method
50 self.n_rankings= math.ceil(number_remove/batches_size)
51 self.edges_to_remove= []
52 self.all_possible_edges= {(u,v) for u in self.graph.nodes for v in range(1, u)}
53
54 def execute_experiment(self):
55     """
56     Método que se encarga de ejecutar el experimento a lo largo de los diferentes
57     ↪ instantes
58     de tiempo determinados por el número de aristas a eliminar y el tamaño de los
59     lotes en los que queremos insertarlas. El objetivo de esta función es almacenar
60     los diferentes rankings que se dan tras la inserción de las aristas por lotes,
61     ↪ para
62     que éstos puedan ser analizados a posteriori a través de otras métricas.
63
64     Returns
65     -----
66     None.
67
68     """
69     inserted_edges= []
70     rankings_list= [_ for _ in range(self.n_rankings)]
71     graph_removed= self.graph.copy()
72     self.edges_to_remove= random.sample(list(self.graph.edges), self.number_remove)
73     ↪ #se sobreescribe
74     graph_removed.remove_edges_from(self.edges_to_remove)
75     self.edges_to_remove= [(edge[0], edge[1]) if edge[0]> edge[1] else (edge[1],
76     ↪ edge[0]) for edge in self.edges_to_remove ]
77     rankings_list[0]= self.compute_ranking(graph_removed)
78     for i in range(self.n_rankings-1): #aquí iteramos sobre el número de inserciones
79         edges_to_insert= self.compute_insertion(rankings_list[i]) #quiero que me
80         ↪ devuelva un set
81         inserted_edges.extend(edges_to_insert)
82         graph_removed.add_edges_from(edges_to_insert)
83         rankings_list[i+1]= self.compute_ranking(graph_removed)
84     edges_to_insert= self.compute_insertion(rankings_list[i+1])

```

```

77     inserted_edges.extend(edges_to_insert)
78     first_ranking= list(rankings_list[0].keys())[: self.number_remove]
79     precision1, precision2= self.precision_evaluation(self.edges_to_remove,
80     ↪ first_ranking, inserted_edges)
81     return (precision1, precision2)
82
83 def compute_ranking(self, graph_removed):
84     """
85     Función que me computa el ranking del índice de interacción correspondiente
86     a las aristas eliminadas del grafo en función de cuál sea la medida de similitud.
87     El objetivo será devolver un OrderedDict que almacene los pares (arista, índice)
88     ↪ en
89     orden ascendente (ya que los índices de inetracción son negativos) de los
90     ↪ índices.
91
92     Parameters
93     -----
94     graph_removed : netwothx Graph or Digraph.
95         Grafo de prueba que no contiene las aristas de estudio.
96     k_index: int.
97         Entero que nos indica la k-vecindad a tener en cuenta a la hora de realizar
98         ↪ los rankings.
99     weighted: boolean.
100         Valor booleano que nos indica si el grafo con el que estamos trabajando es
101         ↪ ponderado o no.
102     removed_edges : list.
103         Lista que contiene tuplas las cuales representan las aristas eliminadas del
104         grafo para el estudio de la predicción sobre ellas.
105     similarity_measure : String.
106         Cadena de texto que nos indica la medida de similitud que queremos utilizar.
107
108     Returns
109     -----
110     ranking: OrderedDict.
111         Diccionario ordenado de aristas según su índice de interacción.
112     """
113     ranking= {}
114     existent_links= {(u, v) if u> v else (v, u) for u, v in graph_removed.edges}
115     non_existent_links= list(self.all_possible_edges - existent_links)
116     if self.similarity_measure== 'SVII':
117         tools.k_neighbors(graph_removed, self.k_index, self.weighted)
118         interaction_index= first_interaction_index.shapley_II_extended(graph_removed)
119         for edge in non_existent_links:
120             ranking[edge]= interaction_index[edge[0]-1][edge[1]-1] if edge[0]>
121             ↪ edge[1] else interaction_index[edge[1]-1][edge[0]-1]
122     elif self.similarity_measure== 'SVMII':
123         tools.subconjuntos(graph_removed, self.k_index, self.weighted)
124         interaction_index=
125         ↪ first_interaction_index.tarkowski_shapley_II_extended(graph_removed)
126         for edge in non_existent_links:
127             ranking[edge]= interaction_index[edge[0]-1][edge[1]-1] if edge[0]>
128             ↪ edge[1] else interaction_index[edge[1]-1][edge[0]-1]
129         #la comprobación que se hace aquí de que el primer vértice de la arista
130         ↪ sea mayor al otro ya no haría falta

```

```

123     elif self.similarity_measure== 'CNN':
124         tools.k_neighbors(graph_removed, self.k_index, self.weighted)
125         for edge in non_existent_links:
126             ranking[edge]=
127                 ↪ first_interaction_index.common_neighbors_similarity(graph_removed,
128                 ↪ edge[0], edge[1])
129     elif self.similarity_measure== 'AAI':
130         tools.k_neighbors(graph_removed, self.k_index, self.weighted)
131         for edge in non_existent_links:
132             ranking[edge]= first_interaction_index.adamic_adar_index(graph_removed,
133                 ↪ edge[0], edge[1])
134     else:
135         raise ValueError("La métrica de similitud insertada no es soportada elija
136             ↪ entre SVII, SVMII, CNN, AAI")
137     return OrderedDict(sorted(ranking.items(), key= lambda x: x[1]))
138
139 def precision_evaluation(self, edges_to_remove, first_ranking, inserted_edges):
140     """
141     Método de la clase que nos devuelve la precisión del primer ranking y la
142     ↪ precisión de las aristas insertadas
143     a través de la reevaluación de la clasificación.
144
145     Returns
146     -----
147     p1 : float.
148         Precisión de la primera clasificación dada.
149     p2 : float.
150         Precisión de las aristas insertadas a través de la reevaluación.
151
152     """
153     n= len(edges_to_remove)
154     print("Imprimimos aristas borradas")
155     print(edges_to_remove)
156     print("Imprimimos primeras aristas recomendadas")
157     print(first_ranking)
158     print("Imprimimos aristas insertadas al final")
159     print(inserted_edges)
160     precision_ranking, precision_reranking= 0, 0
161     for edge in edges_to_remove:
162         precision_ranking+=1 if edge in first_ranking else 0
163         precision_reranking+=1 if edge in inserted_edges else 0
164     p1= (precision_ranking/n)*1000
165     p2= (precision_reranking/n)*1000
166     return (p1, p2)
167
168 def compute_insertion(self, ranking):
169     """
170     Función que elige batches_size aristas del ranking insertado siguiendo el método
171     de selección pasado por parámetro.
172
173     Parameters
174     -----
175     ranking : OrderedDict.
176         DESCRIPTION.

```

```

173         batches_size : int.
174         Entero que nos define el tamaño de los lotes a insertar en nuestra red.
175     method : String.
176         Cadena de texto que nos dice cuál es el método a seguir para elegir
177         ↪ batches_size
178         aristas a insertar en el graph_removed. Nos encontramos con las siguientes
179         ↪ opciones:
180             - random: Elige batches_size aristas al azar del ranking.
181             - score: Elige las batches_size primeras aristas del ranking.
182             - neighborhood: Elige las batches_size aristas más cercanas.
183
184     Returns
185     -----
186     edges_to_insert: set.
187         Conjunto de aristas a insertar en graph_removed
188     """
189     edges= []
190     if self.method== 'random':
191         edges= random.sample(list(ranking.values()), self.batches_size)
192     elif self.method== 'score':
193         edges= list(ranking.keys())[:self.batches_size]
194     else:
195         raise ValueError("El método de selección de aristas no es soportado por favor
196         ↪ elija entre random o score.")
197     return set(edges)
198
199     """
200     graph_zachary= link_prediction.read_data("zachary")
201     zachary= lp_second_experiment(graph_zachary, 3, 12, 3, 'SVII', 'score')
202     zachary.execute_experiment()
203
204     graph_football= link_prediction.read_data("football")
205     football= lp_second_experiment(graph_football, 3, 35, 1, 'SVMII', 'score')
206     football.execute_experiment()
207
208     graph_pequeño= link_prediction.read_data("pequeño")
209     pequeño= lp_second_experiment(graph_pequeño, 2, 3, 1, 'SVMII', 'score')
210     pequeño.execute_experiment()
211
212     graph_zachary= link_prediction.read_data("zachary")
213     zachary= lp_second_experiment(graph_zachary, 2, 9, 1, 'SVII', 'score')
214     zachary.execute_experiment()
215
216     """
217
218 def study_reranking(k_index, percentage, batches_size, similarity_measure, method=
219 ↪ 'score', weighted= False):
220     """
221
222     Parameters
223     -----
224     k_index : TYPE
225         DESCRIPTION.

```

```

224     percentage : TYPE
225         DESCRIPTION.
226     batches_size : TYPE
227         DESCRIPTION.
228     similarity_measure : TYPE
229         DESCRIPTION.
230     method : TYPE, optional
231         DESCRIPTION. The default is 'score'.
232     weighted : TYPE, optional
233         DESCRIPTION. The default is False.
234
235     Returns
236     -----
237     results : TYPE
238         DESCRIPTION.
239
240     """
241     beginning= time.perf_counter()
242     graph_zachary= tools.read_data("zachary")
243     graph_dolphins= tools.read_data("dolphins")
244     graph_football= tools.read_data("football")
245     graph_polbooks= tools.read_data("polbooks")
246     zachary_experiment= lp_second_experiment(graph_zachary, k_index,
247     ↪ round(percentagem*len(graph_zachary.edges)), batches_size, similarity_measure,
248     ↪ method, weighted)
249     dolphins_experiment= lp_second_experiment(graph_dolphins, k_index,
250     ↪ round(percentagem*len(graph_dolphins.edges)), batches_size, similarity_measure,
251     ↪ method, weighted)
252     polbooks_experiment= lp_second_experiment(graph_polbooks, k_index,
253     ↪ round(percentagem*len(graph_polbooks.edges)), batches_size, similarity_measure,
254     ↪ method, weighted)
255     football_experiment= lp_second_experiment(graph_football, k_index,
256     ↪ round(percentagem*len(graph_football.edges)), batches_size, similarity_measure,
257     ↪ method, weighted)
258     p1_zachary, p2_zachary, p1_dolphins, p2_dolphins, p1_polbooks, p2_polbooks,
259     ↪ p1_football, p2_football= np.zeros(20), np.zeros(20), np.zeros(20), np.zeros(20),
260     ↪ np.zeros(20), np.zeros(20), np.zeros(20), np.zeros(20)
261     for i in range(20):
262         p1_zachary[i], p2_zachary[i]= zachary_experiment.execute_experiment()
263         p1_dolphins[i], p2_dolphins[i]= dolphins_experiment.execute_experiment()
264         p1_polbooks[i], p2_polbooks[i]= polbooks_experiment.execute_experiment()
265         p1_football[i], p2_football[i]= football_experiment.execute_experiment()
266     mean_first_precision= [np.mean(p1_zachary), np.mean(p1_dolphins),
267     ↪ np.mean(p1_polbooks), np.mean(p1_football)]
268     mean_second_precision= [np.mean(p2_zachary), np.mean(p2_dolphins),
269     ↪ np.mean(p2_polbooks), np.mean(p2_football)]
270     results= pd.DataFrame({
271         "Dataset": ['Zachary', 'Dolphins', 'Polbooks', 'Football'],
272         "First precision": mean_first_precision,
273         "Second precision": mean_second_precision})
274     end= time.perf_counter()
275     execution_time= end-beginning
276     print(f"The execution time was {execution_time:.5f} segundos")
277     return results

```

```

267
268 def precision_evaluation(edges_to_remove, first_ranking, inserted_edges):
269     """
270     Método de la clase que nos devuelve la precisión del primer ranking y la precisión
271     ↪ de las aristas insertadas
272     a través de la reevaluación de la clasificación.
273
274     Returns
275     -----
276     p1 : float.
277         Precisión de la primera clasificación dada.
278     p2 : float.
279         Precisión de las aristas insertadas a través de la reevaluación.
280
281     """
282     n= len(edges_to_remove)
283     precision_ranking, precision_reranking= 0, 0
284     for edge in edges_to_remove:
285         precision_ranking+=1 if edge in first_ranking else 0
286         precision_reranking+=1 if edge in inserted_edges else 0
287     p1= (precision_ranking/n)*1000
288     p2= (precision_reranking/n)*1000
289     return (p1, p2)

```

A.4. Problema de detección de comunidades

Este script realiza el procedimiento de detección de comunidades, para ello contiene una clase para el almacenamiento de Clusters y otra para la realización del experimento. De manera adicional, creamos funciones para el cálculo del coverage y para ejecutar los algoritmos Greedy y Louvain sobre los conjuntos de datos.

```

1  import numpy as np
2  from time import time
3  import math
4  from matplotlib import pyplot as plt
5  from scipy.cluster.hierarchy import dendrogram
6  import sys
7  import pandas as pd
8  import networkx as nx
9  import tools
10 import first_interaction_index
11 from tools import read_data
12 import community as community_louvain
13 from collections import defaultdict
14
15
16 class Cluster:
17     ClusterCount = 0
18
19     def __init__(self, nodes= []):
20         """
21         Método de inicialización de un elemento de la clase Cluster. Una vez generamos
22         ↪ una nueva instancia de esta

```

```

22     clase se actualiza el número de clusters que existen, a través del atributo de
    ↪ clase ClusterCount. De esta manera,
23 Cluster Count nos define el número de clusters que se han creado de la clase, y
    ↪ hemos de tener en cuenta que los
24 identificadores de los clusters irán de 1 hasta ClusterCount.
25
26     Parameters
27     -----
28     nodes : list.
29         Lista que contiene los identificadores de los nodos que se encuentran en
    ↪ este cluster. Su valor por
30 defecto es [].
31
32     """
33     Cluster.ClusterCount+=1
34     self.id= Cluster.ClusterCount
35     self.nodes= nodes
36
37     def add_node(self, node_id):
38         """
39         Método de la clase que añade un nuevo nodo al cluster.
40
41         Pameters
42         -----
43         node_id : int.
44         Identificador del nodo que se quiere añadir a este nuevo cluster.
45         """
46         self.nodes.append(node_id)
47
48
49     class clustering_experiment():
50
51         def __init__(self, graph, k_index, weighted, similarity_measure):
52             """
53             Método de inicialización del experimento. Este inicializa los atributos que van a
    ↪ ser necesarios para el desarrollo del experimento.
54             Veamos cuáles son:
55             - graph: El grafo en el que queremos encontrar una separación por
    ↪ comunidades.
56             - k_index: El radio de la vecindad que tendremos en cuenta para calcular las
    ↪ similitudes entre nodos.
57             - weighted: Booleano que nos dirá si es ponderado o no.
58             - similarity_measure: Nos indica la métrica con la que queremos calcular el
    ↪ índice de interacción entre nodos.
59             - modularity: será un array de numpy que almacenará la modularidad de cada
    ↪ configuración.
60             - configurations: array de numpy que almacenará en cada fila las etiquetas
    ↪ que le corresponden a cada uno de los nodos en la configuración actual
61             - linkage: será la matriz de fusiones de clusters que se pasará como
    ↪ parámetro de entrada a dendrograma de scipy.
62             - clusters: diccionario que almacena todos los clusters generados en el
    ↪ proceso, a cada identificador de cluster le da el objeto de la clase
    ↪ Cluster
63             - similarity_matrix: matriz original de índices de interacción entre los
    ↪ nodos del grafo, es simétrica con 0s en la diagonal.

```

```

64         - similarity_df: dataframe que va almacenando los índices de interacción
        ↪ entre las comunidades actuales.
65
66     Parameters
67     -----
68     graph : TYPE
69         DESCRIPTION.
70     k_index : TYPE
71         DESCRIPTION.
72     weighted : TYPE
73         DESCRIPTION.
74     similarity_measure : TYPE
75         DESCRIPTION.
76
77     Returns
78     -----
79     None.
80
81     """
82     Cluster.ClusterCount= 0
83     self.graph= graph
84     self.k_index= k_index
85     self.weighted= weighted
86     self.similarity_measure= similarity_measure
87     self.modularity= np.zeros(len(graph.nodes), dtype= float)
88     self.configurations= np.zeros((len(graph.nodes), len(graph.nodes)), dtype= int)
89     self.linkage= np.zeros((len(graph.nodes)-1, 4), dtype= float)
90     self.clusters= {}
91     for node in range(1, len(graph.nodes)+1):
92         cluster= Cluster([node])
93         self.clusters[cluster.id]= cluster
94     if similarity_measure== 'SVII':
95         self.similarity_matrix= np.zeros((len(graph.nodes), len(graph.nodes)), dtype=
        ↪ np.float64)
96         tools.k_neighbors(graph, k_index, weighted)
97         M= first_interaction_index.shapley_II_extended(graph)
98     elif similarity_measure== 'SVMII':
99         self.similarity_matrix= np.zeros((len(graph.nodes), len(graph.nodes)), dtype=
        ↪ np.float64)
100         tools.subconjuntos(graph, k_index, weighted)
101         M= first_interaction_index.tarkowski_shapley_II_extended(graph)
102     elif similarity_measure== 'CNN':
103         self.similarity_matrix= np.zeros((len(graph.nodes), len(graph.nodes)), dtype=
        ↪ np.float64)
104         tools.k_neighbors(graph, k_index, weighted)
105         M= first_interaction_index.common_neighbors_extended(graph)
106     elif similarity_measure== 'AAI':
107         self.similarity_matrix= np.zeros((len(graph.nodes), len(graph.nodes)), dtype=
        ↪ np.float64)
108         tools.k_neighbors(graph, k_index, weighted)
109         M= first_interaction_index.adamic_adar_extended(graph)
110     else:
111         raise ValueError("La medida de similitud introducida no es reconocida,
        ↪ porfavor seleccione una entre: SVII, SVMII, CNN")
112     self.similarity_matrix = M + M.T

```

```

113     self.similarity_df= pd.DataFrame(self.similarity_matrix.copy())
114     self.similarity_df.rename(index= lambda x: x+1, columns= lambda x: x+1, inplace=
    ↪ True)
115
116
117 def merging_clusters(self):
118     """
119     Método de la clase que nos identifica cuáles son las siguientes dos comunidades
120     a fusionar.
121
122     Returns
123     -----
124     clusterA, clusterB : int
125         Identificadores de las comunidades que queremos fusionar.
126     minimum_II : float.
127         Índice de interacción entre las comunidades.
128
129     """
130
131     minimum_II= math.inf
132     clusterA, clusterB= None, None
133     for row in range(0, self.similarity_df.shape[0]-1):
134         for column in range(row+1, self.similarity_df.shape[1]):
135             if self.similarity_df.iloc[row, column]< minimum_II:
136                 minimum_II= self.similarity_df.iloc[row, column]
137                 clusterA, clusterB= row, column
138     return clusterA, clusterB, minimum_II
139
140 def merge_communities(self, clusterA_id, clusterB_id):
141     """
142     Método de la clase cluster experiment que realiza el proceso de fusión de
    ↪ clusters. Para ello,
143     lo que hace es: crear una nueva instancia de la clase clusters y ajustar la
    ↪ matriz de similitudes
144     para reflejar correctamente las nuevas relaciones entre el nuevo cluster y el
    ↪ resto.
145
146     Parameters
147     -----
148     clusterA_id : Objeto inmutable.
149         Identificador o índice/nombre de columna del primero de los clusters que
    ↪ queremos unificar.
150     clusterB_id : Objeto inmutable.
151         Identificador o índice/nombre de columna del segundo de los clusters que
    ↪ queremos unificar.
152
153     Returns
154     -----
155     new_cluster : Cluster() element.
156         Nuevo cluster creado a raíz de la fusión de los otros dos.
157
158     """
159     new_cluster= Cluster(self.clusters[clusterA_id].nodes +
    ↪ self.clusters[clusterB_id].nodes)
    self.clusters[new_cluster.id]= new_cluster

```

```

160     self.similarity_df[clusterA_id]=
    ↪     self.similarity_df[clusterA_id].combine(self.similarity_df[clusterB_id], max)
161     self.similarity_df.loc[clusterA_id]=
    ↪     self.similarity_df.loc[clusterA_id].combine(self.similarity_df.loc[clusterB_id],
    ↪     max)
162     self.similarity_df.drop(index= clusterB_id, inplace= True)
163     self.similarity_df.drop(columns= clusterB_id, inplace= True)
164     self.similarity_df.rename(index= {clusterA_id: new_cluster.id}, inplace= True)
165     self.similarity_df.rename(columns= {clusterA_id: new_cluster.id}, inplace= True)
166     print(self.similarity_df)
167     return new_cluster
168
169 def compute_modularity(self, configuration): #no hace falta pasarle el grafo porque
    ↪ ya lo tiene al alcance
170     """
171     Método de la clase que calcula la modularidad de una configuración dada. Este
    ↪ método recibe una configuración por clusters
172     a través de un numpy array y devuelve la modularidad que le corresponde haciendo
    ↪ uso del método modularity de la librería
173     networkx.
174
175     Parameters
176     -----
177     configuration: np.array.
178         Array de numpy de una dimensión que contiene para cada nodo el identificador
    ↪ del cluster al que pertenece en la
179         configuración actual.
180
181     Returns
182     -----
183     La modularidad de la configuración.
184     """
185     labels= set(configuration)
186     clusters= []
187     for label in labels:
188         community= set()
189         for i in range(len(configuration)):
190             if configuration[i]== label:
191                 community.add(i+1)
192             clusters.append(community)
193     return nx.algorithms.community.quality.modularity(self.graph, clusters)
194
195 def execute_experiment(self):
196     """
197     Función de la clase que se encarga de ejecutar el experimento.
198
199     Returns
200     -----
201     max_modularity : float.
202         El valor de la modularidad máxima conseguida a través del proceso de
    ↪ clustering aglomerativo.
203     max_configuration : numpy array de una dimensión.
204         Array que contiene para cada posición, la cual define un nodo en orden
    ↪ correspondiente (1, 2, .., n) el cluster asignado
205         en la configuración con mayor modularidad.

```

```

206         """
207
208     self.configurations[0, :] = np.arange(1, len(self.graph.nodes)+1)
209     self.modularity[0] = self.compute_modularity(self.configurations[0, :])
210     for i in range(1, self.graph.number_of_nodes()):
211         clusterA, clusterB, minimum_II = self.merging_clusters()
212         print("Imprimimos el indice de interaccion menor")
213         print(clusterA, clusterB, minimum_II)
214         clusterA_id, clusterB_id = self.similarity_df.index[clusterA],
215         ↪ self.similarity_df.index[clusterB]
216         print(clusterA_id, clusterB_id)
217         new_cluster = self.merge_communities(clusterA_id, clusterB_id)
218         print("Imprimimos el identificador del nuevo cluster")
219         print(new_cluster.id)
220         distancia = 4 if minimum_II == 0 else -1/minimum_II
221         self.linkage[i-1] = np.array([int(clusterA_id)-1, int(clusterB_id)-1,
222         ↪ distancia, len(new_cluster.nodes)]) #actualizamos la matriz de linkage
223         previous = self.configurations[i-1]
224         new_configuration = np.where((previous == clusterA_id) | (previous ==
225         ↪ clusterB_id), new_cluster.id, previous)
226         self.configurations[i] = new_configuration
227         self.modularity[i] = self.compute_modularity(new_configuration)
228     max_modularity = np.max(self.modularity)
229     max_configuration = self.configurations[np.argmax(self.modularity)]
230     max_coverage = compute_coverage(self.graph, max_configuration)
231     plt.figure(figsize=(8,5))
232     etiquetas = [str(i) for i in range(1, len(self.graph.nodes)+1)]
233     dendrogram(self.linkage, orientation='right', labels=etiquetas)
234     plt.show()
235     return max_modularity, max_configuration, max_coverage
236
237 def compute_coverage(graph, configuration):
238     """
239     Método que calcula el coverage de una clusterización cualquiera de un grafo.
240
241     Parameters
242     -----
243     graph: Networkx Graph.
244         Grafo de la librería Networkx.
245     configuration : numpy array.
246         Array de numpy que contiene en cada posición la comunidad a la que pertenece el
247         ↪ nodo con
248         identificador la posición en el array +1.
249
250     Returns
251     -----
252     float
253         Medida de coverage de la división en comunidades del grafo actual.
254
255     """
256     node_community = {}
257     for i in range(len(configuration)):
258         node_community[i+1] = configuration[i]

```

```

257     internal_edges= sum(1 for u, v in graph.edges() if node_community[u]==
    ↪ node_community[v])
258     return internal_edges/(graph.number_of_edges())
259
260
261 def greedy_louvain_modularity():
262     graphs= [read_data("zachary"), read_data("dolphins"), read_data("polbooks"),
    ↪ read_data("football")]
263     mod_greedy, number_greedy, mod_louvain, number_louvain= [], [], [], []
264     for i in range(4):
265         communities_greedy = nx.community.greedy_modularity_communities(graphs[i])
266         communities_greedy= [list(c) for c in communities_greedy]
267         partition= community_louvain.best_partition(graphs[i])
268         communities_louvain = defaultdict(set)
269         for node, com in partition.items():
270             communities_louvain[com].add(node)
271         communities_louvain= list(communities_louvain.values())
272         mod_greedy.append(nx.community.modularity(graphs[i], communities_greedy))
273         number_greedy.append(len(communities_greedy))
274         mod_louvain.append(nx.community.modularity(graphs[i], communities_louvain))
275         number_louvain.append(len(communities_louvain))
276     results= pd.DataFrame({
277         "Dataset": ['Zachary', 'Dolphins', 'Polbooks', 'Football'],
278         "Greedy MOD": mod_greedy,
279         "Greedy n° communities": number_greedy,
280         "Louvain MOD": mod_louvain,
281         "Louvain n° communities": number_louvain
282     })
283     return results

```